

Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Lic. en Sistemas de Información

TP N.° 3.2 — Programación de Sockets

Redes de Datos

2026

Alumno: Orban, Tobias Naim
DNI: 46.385.637
LU: 58009

Índice

1. Entorno de trabajo **3**

2. Ejercicio 1 — Chat mediante sockets **4**

2.1. Arquitectura de la solución 4

2.2. Protocolo de aplicación 4

2.3. Librerías importadas 5

2.4. Servidor: creación y uso de los sockets 5

2.5. Cliente: conexión y comunicación 6

2.6. Estrategia de concurrencia 7

2.7. Interfaz de usuario y comandos especiales 8

2.8. Manejo de errores y desconexiones 8

2.9. Pruebas de funcionamiento 9

3. Ejercicio 2 — Servidor de consultas de HW/SW para inventario **12**

3.1. Diseño y roles 12

3.2. Protocolo de aplicación 12

3.3. Consultas implementadas 12

3.4. Comunicación por sockets: líneas relevantes 13

3.5. Concurrencia y manejo de errores 14

3.6. Pruebas de funcionamiento 14

3.7. Comparación con UDP 16

4. Ejercicio 3 — Inspección en Wireshark **17**

4.1. Condiciones de la captura 17

4.2. Identificación de clientes y tuplas de cada conexión 17

4.3. Establecimiento de las conexiones 20

4.4. Flujo de datos no cifrado 20

4.5. Retransmisión de los mensajes por el servidor 21

4.6. Mensaje con la LU 22

4.7. Cierre de las conexiones 22

5. Conclusiones **24**

Anexo A — Código fuente del Ejercicio 1 **25**

A.1. servidor_chat.c 25

A.2. cliente_chat.c 30

Anexo B — Código fuente del Ejercicio 2 **36**

B.1. servidor_inventario.c 36

B.2. agente_inventario.c 42

1. Entorno de trabajo

El práctico se desarrolló sobre una máquina virtual Linux ejecutada en Oracle VM VirtualBox en el equipo anfitrión. El servidor y los clientes se ejecutaron como procesos independientes dentro de la máquina virtual, cada uno en su propia sesión de terminal: la consola de VirtualBox y sesiones SSH abiertas desde el anfitrión con PuTTY. Cada sesión ejecuta su propio proceso cliente, con su propio socket y su propio puerto de origen, por lo que el servidor las trata como conexiones totalmente independientes.

Parámetro	Valor
Equipo anfitrión	Microsoft Windows 11
Virtualización	Oracle VM VirtualBox, máquina virtual «Ubuntu-Redes»
Sistema operativo de la VM	Ubuntu Server (hostname `ubuntu-server`)
Interfaz de red de la VM	`enp0s3` — 192.168.1.2, red local 192.168.1.0/24
Lenguaje y compilación	C, compilado con GCC mediante `make` (paquete build-essential)
Interfaz de programación	Sockets de Berkeley (POSIX), TCP sobre IPv4; hilos POSIX (pthreads)
Puertos de servicio	5000/TCP (chat, Ejercicio 1) y 6000/TCP (inventario, Ejercicio 2)
Captura de tráfico	`tcpdump` en la VM (archivo `chat.pcap`), analizado con Wireshark en el anfitrión

El código se organizó en una carpeta por ejercicio, con un `Makefile` que compila los cuatro programas con las opciones `-Wall -Wextra -O2 -pthread`. La compilación finaliza sin advertencias:

```
TP3.2/
├── Makefile
├── ej1/servidor_chat.c        (498 líneas)  servidor de chat
├── ej1/cliente_chat.c         (408 líneas)  cliente de chat
├── ej2/servidor_inventario.c  (575 líneas)  servidor de inventario
└── ej2/agente_inventario.c    (641 líneas)  agente de inventario

$ unzip TP3.2.zip && cd TP3.2 && make
```

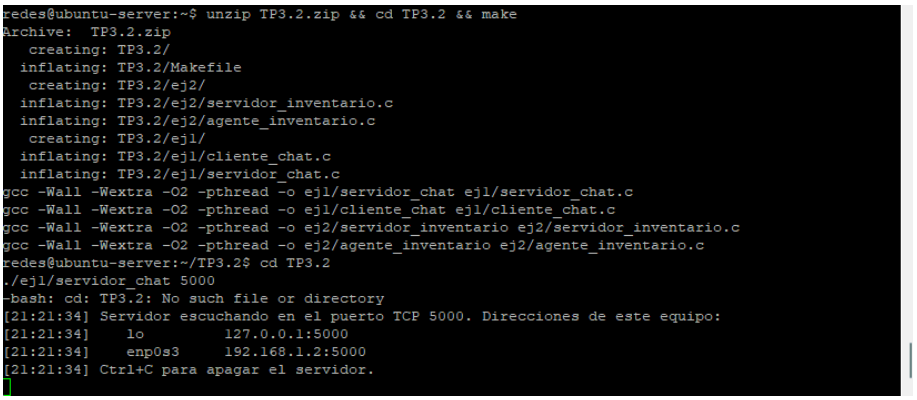


Figura 1. *Descompresión y compilación de los cuatro programas con make en la máquina virtual, y arranque del servidor de chat, que informa las direcciones en las que escucha.*

Se eligieron los puertos 5000 y 6000 por pertenecer al rango de puertos registrados (1024–49151): a diferencia de los puertos bien conocidos (0–1023), no requieren privilegios de superusuario para el `bind()` y no estaban en uso en la máquina virtual. Ambos programas servidores aceptan un puerto alternativo como argumento (`./servidor_chat 5001`).

2. Ejercicio 1 — Chat mediante sockets

2.1. Arquitectura de la solución

Se implementó un chat de arquitectura cliente-servidor con un **servidor central**: los clientes no se comunican entre sí directamente, sino que cada uno mantiene una única conexión TCP con el servidor, y este retransmite cada mensaje recibido a todos los demás clientes conectados.

- **servidor_chat.c** (498 líneas): crea un socket de escucha en el puerto 5000 y, por cada cliente aceptado, un socket conectado atendido por un hilo propio. Mantiene la lista de usuarios, retransmite los mensajes, resuelve los comandos y notifica los ingresos y egresos.
- **cliente_chat.c** (408 líneas): solicita IP y puerto del servidor, informa si la conexión fue exitosa, registra el nombre de usuario y luego trabaja con dos hilos: uno lee el teclado y envía, el otro recibe y muestra los mensajes en tiempo real.

Se utilizó **TCP** (`SOCK_STREAM`) como protocolo de transporte. Un chat necesita que los mensajes lleguen completos, sin pérdidas y en el orden en que se escribieron, garantías que TCP ofrece de forma nativa. Además, al ser orientado a conexión, el servidor sabe en todo momento qué clientes están conectados y detecta su desconexión: `recv()` devuelve 0 ante un cierre ordenado o un error ante un corte abrupto. Con UDP habría que reconstruir estas funciones en la propia aplicación.

2.2. Protocolo de aplicación

TCP entrega un **flujo de bytes** sin límites entre mensajes: un `recv()` puede devolver media línea o varias líneas juntas. Por eso se definió un protocolo de texto en el que **cada mensaje es una línea terminada en** `\n`. La función `leer_linea()` (servidor, líneas 121–151) acumula lo recibido en un buffer propio de cada conexión hasta encontrar el salto de línea, y recién entonces entrega la línea completa.

Sentido	Mensaje	Significado
Cliente → Servidor	`<nombre>`	Primera línea: nombre de usuario a registrar
Cliente → Servidor	`<texto>`	Mensaje para todos los participantes
Cliente → Servidor	`/listar /quitar /ayuda`	Comandos especiales
Servidor → Cliente	`OK <nombre>`	Nombre aceptado; el cliente queda registrado
Servidor → Cliente	`MSG <usuario>: <texto>`	Mensaje de otro usuario, retransmitido por el servidor
Servidor → Cliente	`SYS <texto>`	Aviso del sistema: bienvenida, ingresos, egresos, apagado
Servidor → Cliente	`LST <texto>`	Respuesta al comando /listar
Servidor → Cliente	`ERR <texto>`	Error: nombre inválido o repetido, comando desconocido

Las etiquetas de tres letras permiten al cliente dar a cada tipo de mensaje un color distinto. Al ser texto plano con un delimitador simple, cualquier programa que respete el formato puede interoperar con el servidor, independientemente del lenguaje en que esté escrito, como pide el tercer objetivo del práctico.

2.3. Librerías importadas

Cabecera	Uso en el programa
`<sys/socket.h>`	Primitivas de sockets: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`, `setsockopt()`, `shutdown()`; constantes `AF_INET`, `SOCK_STREAM`.
`<netinet/in.h>`	Estructura `sockaddr_in` (dirección IPv4 + puerto) y la constante `INADDR_ANY`.
`<arpa/inet.h>`	Conversión de orden de bytes (`htons()`, `htonl()`, `ntohs()`) y de direcciones a texto (`inet_ntop()`).
`<netdb.h>`	Solo cliente: `getaddrinfo()` resuelve la IP o el nombre del servidor ingresado por el usuario.
`<unistd.h>`	`close()` para liberar los sockets.
`<pthread.h>`	Hilos (`pthread_create()`, `pthread_detach()`, `pthread_join()`) y exclusión mutua (`pthread_mutex_*`).
`<fcntl.h>`, `<sys/select.h>`	Solo cliente: `connect()` no bloqueante con tiempo máximo de espera.
`<ifaddrs.h>`	Solo servidor: `getifaddrs()` lista las IP locales para mostrarlas al iniciar.
`<signal.h>`, `<errno.h>`	Manejo de `SIGPIPE` y `SIGINT`, y de los códigos de error (`EADDRINUSE`, `ECONNREFUSED`, etc.).

Las inclusiones se encuentran en las líneas 23–38 del servidor y 15–29 del cliente. No se utilizan bibliotecas externas: todo pertenece a la biblioteca estándar de C y a POSIX.

2.4. Servidor: creación y uso de los sockets

El servidor sigue la secuencia pasiva clásica **socket** → **bind** → **listen** → **accept**. Cada `accept()` devuelve un **socket nuevo**, exclusivo para ese cliente, mientras el socket de escucha sigue disponible para aceptar otras conexiones.

```
409     /* (1) socket(): socket IPv4 (AF_INET) de tipo flujo (SOCK_STREAM) = TCP */
410     fd_escucha = socket(AF_INET, SOCK_STREAM, 0);
411     if (fd_escucha < 0) {
412         perror("socket");
413         return 1;
414     }
415
416     /* Permite reutilizar el puerto enseguida tras reiniciar el servidor
417      * (evita "Address already in use" por conexiones en TIME_WAIT). */
418     int si = 1;
419     setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof si);
421     /* (2) bind(): asocia el socket a todas las interfaces y al puerto elegido */
422     struct sockaddr_in dir;
423     memset(&dir, 0, sizeof dir);
424     dir.sin_family      = AF_INET;
425     dir.sin_addr.s_addr = htonl(INADDR_ANY);  /* 0.0.0.0: cualquier IP local */
426     dir.sin_port        = htons((uint16_t)puerto);  /* orden de red (big endian) */
427     if (bind(fd_escucha, (struct sockaddr *)&dir, sizeof dir) < 0) {
440     /* (3) listen(): el socket pasa a modo pasivo y espera conexiones */
441     if (listen(fd_escucha, BACKLOG) < 0) {
448     /* (4) accept(): bucle que acepta clientes y crea un hilo para cada uno */
449     while (!apagando) {
450         struct sockaddr_in cli;
451         socklen_t lc = sizeof cli;
452         int fd = accept(fd_escucha, (struct sockaddr *)&cli, &lc);
```

Línea	Instrucción	Función
410	`socket(AF_INET, SOCK_STREAM, 0)`	Crea el socket de escucha. `AF_INET` indica IPv4 y `SOCK_STREAM` un flujo orientado a conexión, que el núcleo implementa con TCP (el protocolo 0 selecciona el predeterminado para esa combinación).

Línea	Instrucción	Función
419	`setsockopt(..., SO_REUSEADDR)`	Permite volver a enlazar el puerto aunque queden conexiones anteriores en estado TIME_WAIT, para reiniciar el servidor sin esperar.
424–427	`bind()`	Asocia el socket a 0.0.0.0:5000. `INADDR_ANY` acepta conexiones por cualquier interfaz (lo y enp0s3); `htonl()` y `htons()` pasan dirección y puerto al orden de bytes de la red (big-endian).
441	`listen(fd_escucha, BACKLOG)`	Pasa el socket a modo pasivo. `BACKLOG` = 10 es la cantidad de conexiones ya establecidas que pueden esperar en cola hasta ser aceptadas.
452	`accept()`	Se bloquea hasta que un cliente completa el three-way handshake y devuelve el socket conectado. En `cli` recibe la IP y el puerto del cliente, convertidos a texto en las líneas 471–472.
463	`setsockopt(fd, SO_SNDTIMEO)`	Limita a 3 s la espera de un `send()`, para que un cliente que no lee no bloquee al servidor.
132	`recv()`	Dentro de `leer_linea()`: recibe bytes del cliente. Devuelve 0 si el cliente cerró la conexión y −1 ante un error (por ejemplo `ECONNRESET` en un corte abrupto).
93	`send(..., MSG_NOSIGNAL)`	Dentro de `enviar_todo()`: se repite hasta enviar todos los bytes. `MSG_NOSIGNAL` evita que el proceso reciba `SIGPIPE` si el cliente ya se desconectó.
229–243	`difundir()`	Retransmisión: recorre la lista de clientes y envía la línea a todos excepto al emisor.
357	`close()`	Cierra el socket del cliente al terminar su hilo.
369, 491	`shutdown()`	Apagado ordenado: interrumpe el `accept()` bloqueado y cierra las conexiones de los clientes.

2.5. Cliente: conexión y comunicación

El cliente solicita por teclado la IP (o el nombre) del servidor y el puerto, con 127.0.0.1 y 5000 como valores predeterminados; también pueden pasarse como argumentos. La conexión se realiza en la función `conectar()` (líneas 218–285):

```
233         /* (1) socket(): crea el socket TCP del cliente */
234         fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
235         if (fd < 0) {
236             err = errno;
237             continue;
238         }
239
240         /* (2) connect(): inicia el three-way handshake (SYN, SYN-ACK, ACK).
241          * Se hace no bloqueante para poder limitar la espera con select(). */
242         int flags = fcntl(fd, F_GETFL, 0);
243         fcntl(fd, F_SETFL, flags | O_NONBLOCK);
244         int r = connect(fd, p->ai_addr, p->ai_addrlen);
245         if (r < 0 && errno == EINPROGRESS) {
246             fd_set escritura;
247             FD_ZERO(&escritura);
248             FD_SET(fd, &escritura);
249             struct timeval tv = { .tv_sec = TIMEOUT_CONEXION, .tv_usec = 0 };
250             r = select(fd + 1, NULL, &escritura, NULL, &tv);
```

Línea	Instrucción	Función
225	`getaddrinfo()`	Traduce el texto ingresado (IP o nombre, por ejemplo «localhost») a una dirección IPv4 con puerto, lista para `connect()`.

Línea	Instrucción	Función
234	`socket()`	Crea el socket TCP del cliente.
243–244	`connect()`	Inicia el three-way handshake con el servidor. En este momento el núcleo asigna al cliente un **puerto de origen efímero**. El socket se pone en modo no bloqueante para poder limitar la espera.
250–256	`select()` + `getsockopt(SO_ERROR)`	Espera como máximo 5 s a que la conexión se complete y obtiene su resultado: éxito, conexión rechazada, host inalcanzable o tiempo agotado.
314–315	`getsockname()`, `getpeername()`	Obtienen las dos puntas de la conexión para mostrar el mensaje «Conexión exitosa: IP:puerto (cliente) → IP:puerto (servidor)», que luego se contrasta con Wireshark.
358	`pthread_create()`	Lanza el hilo receptor.
94	`recv()`	Dentro de `leer_linea()`: recepción de los mensajes del servidor.
63	`send()`	Dentro de `enviar_todo()`: envío de mensajes y comandos.
213	`send()` en manejador de señal	Ante Ctrl+C envía `/quitar` para salir de forma ordenada (`send()` puede usarse dentro de un manejador de señales).
197, 203	`close()`	Cierre del socket al desconectarse.

2.6. Estrategia de concurrencia

Servidor: un hilo por conexión. El hilo principal solo ejecuta el bucle de `accept()`. Por cada cliente crea un hilo con `pthread_create()` (línea 476) que ejecuta `atender_cliente()` (líneas 271–359) y lo desacopla con `pthread_detach()` (línea 482) para que sus recursos se liberen solos al terminar. Cada hilo queda bloqueado en el `recv()` de su propio cliente sin afectar a los demás, de modo que el servidor atiende a todos los clientes en forma concurrente.

Los hilos comparten la lista `clientes[]` (hasta 50 usuarios), protegida con el mutex `mutex_clientes`: registrar un nombre, quitar un usuario, retransmitir un mensaje y armar la lista para `/listar` se hacen siempre con el mutex tomado, lo que evita condiciones de carrera, por ejemplo dos clientes que eligen el mismo nombre en el mismo instante. Un segundo mutex (`mutex_log`) ordena la salida por consola.

Cliente: dos hilos. El hilo principal lee el teclado con `fgets()` y envía; el hilo receptor (`recibir()`, líneas 184–204) permanece bloqueado en `recv()` y muestra cada mensaje apenas llega. Así la recepción nunca espera a que el usuario termine de escribir. El mutex `mutex_pantalla` evita que ambos hilos escriban en la consola al mismo tiempo y redibuja el indicador «Nombre>» después de cada mensaje.

Como alternativa se podía usar un único hilo con `select()` o `poll()` sobre todos los sockets. Se eligió el modelo de hilos por la claridad del código: cada conexión se programa como una secuencia lineal de lecturas y escrituras.

2.7. Interfaz de usuario y comandos especiales

Al iniciar, el cliente pide la IP y el puerto del servidor, informa la conexión y solicita el nombre de usuario. Los mensajes se muestran con la hora y un color según su tipo: el nombre del emisor en cian, los avisos del sistema en amarillo, la lista de usuarios en verde y los errores en rojo. El nombre debe tener entre 1 y 20 caracteres (letras, números, «_», «-» o «.») y no puede repetirse, sin distinguir mayúsculas; se permiten 3 intentos.

Comando	Comportamiento	Líneas (servidor)
`/listar`	El servidor responde con `LST` y la lista de usuarios conectados, marcando al solicitante con «(vos)».	248–264, 325
`/quitar`	El servidor se despide, cierra la conexión y avisa a los demás que el usuario salió; el cliente termina. Ctrl+C y Ctrl+D también envían /quitar.	327–329, 349
`/ayuda`	Muestra los comandos disponibles.	331
otro `/…`	Responde `ERR Comando desconocido`.	334

2.8. Manejo de errores y desconexiones

Situación	Comportamiento	Líneas
Desconexión normal	Con `/quitar` el hilo sale del bucle, quita al cliente de la lista, cierra el socket y notifica «X salió del chat».	S 327, 349
Desconexión abrupta	Si se cierra la terminal o se mata el proceso, `recv()` devuelve 0 o un error (`ECONNRESET`). El servidor libera al cliente, avisa «X se desconectó inesperadamente» y sigue atendiendo al resto.	S 343–353
Escritura a un cliente caído	`SIGPIPE` se ignora y `send()` usa `MSG_NOSIGNAL`: un envío fallido devuelve error en lugar de terminar el proceso servidor.	S 401, 93
Cliente que no lee	`SO_SNDTIMEO` de 3 s evita que un cliente bloqueado detenga la retransmisión.	S 463
Caída o apagado del servidor	Ctrl+C en el servidor interrumpe el `accept()`, avisa a todos «El servidor se está apagando» y cierra las conexiones. El cliente detecta el cierre en `recv()`, informa «Se perdió la conexión con el servidor» y termina.	S 364, 485–494; C 184–204
Puerto ocupado	`bind()` falla con `EADDRINUSE`: se informa el problema y se sugiere otro puerto. `SO_REUSEADDR` evita el falso «ocupado» tras reiniciar.	S 427–436
Datos inválidos	Nombres vacíos, largos, con caracteres no permitidos o repetidos se rechazan con `ERR`. Los caracteres de control se reemplazan, las líneas vacías se ignoran y las demasiado largas se cortan. El cliente no envía mensajes de más de 400 caracteres y valida el puerto.	S 171–182, 156
Servidor lleno o error de `accept()`	Con 50 usuarios se responde `ERR Servidor lleno`; un fallo de `accept()` se registra y el servidor continúa.	S 297, 456
Servidor inexistente o inalcanzable	El cliente distingue conexión rechazada (`ECONNREFUSED`), host inalcanzable y tiempo agotado (5 s), e informa cada caso.	C 275–283

S: servidor_chat.c; *C*: cliente_chat.c.

2.9. Pruebas de funcionamiento

Se inició el servidor en la máquina virtual y se conectaron **tres clientes** desde sesiones distintas («Tobias», «Juancho» y «Matias»), ingresando como servidor la dirección 192.168.1.2 y el puerto 5000. El registro del servidor muestra cada conexión con la IP y el puerto de origen del cliente (192.168.1.2:38674, :40280 y

:40160), el registro de los nombres con la cantidad de conectados, los mensajes recibidos, la salida de cada usuario con /quitar y el apagado ordenado con Ctrl+C. Esta misma sesión es la que se capturó para el Ejercicio 3.

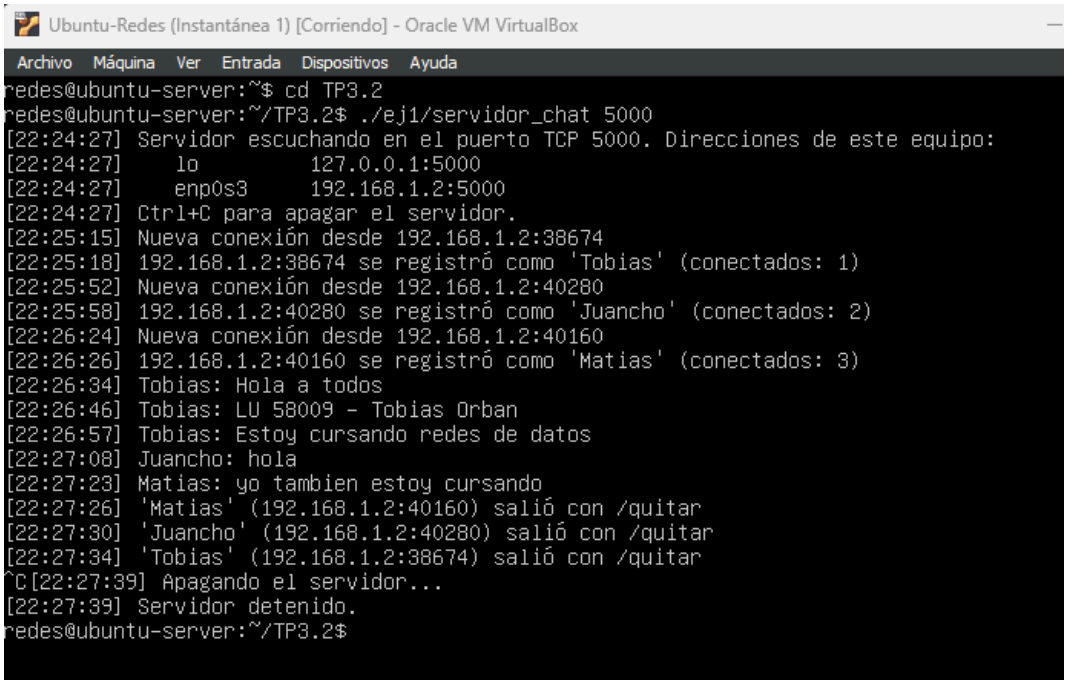


Figura 2. *Registro del servidor de chat con tres clientes concurrentes: conexiones con su IP y puerto de origen, registro de usuarios, mensajes, salidas con /quitar y apagado del servidor.*

En el cliente de Tobias se observa la confirmación de conexión con la tupla completa, los avisos de ingreso de Juancho y Matias, el envío de mensajes (entre ellos el que contiene la LU), los mensajes de los demás en tiempo real, identificados con el nombre del emisor y la hora, los avisos de salida y la despedida del servidor al ejecutar /quitar. Los mensajes propios no vuelven del servidor, porque `difundir()` excluye al emisor.



Figura 3. *Cliente «Tobias»: conexión exitosa, ingresos y salidas de otros participantes, envío y recepción de mensajes en tiempo real y desconexión con /quitar.*

El cliente de Juancho ejecutó `/listar`, que devolvió los tres usuarios conectados marcando al solicitante con «(vos)», y recibió los mensajes de los otros dos clientes.

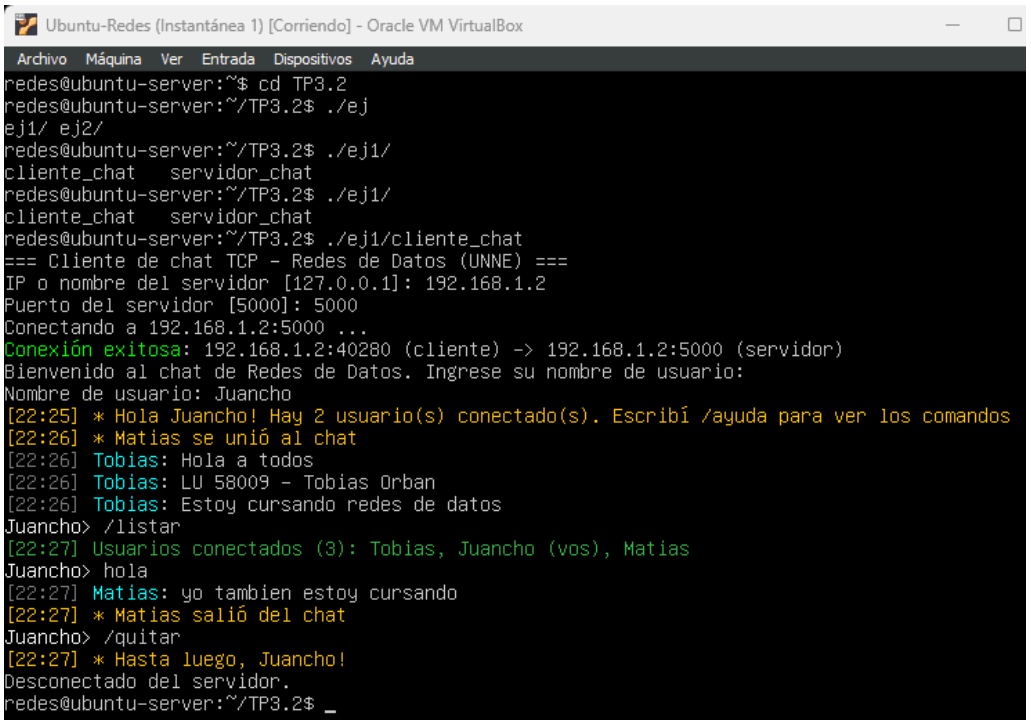


Figura 4. *Cliente «Juancho»: recepción de mensajes de dos emisores distintos, comando /listar y salida con /quitar.*

El cliente de Matias probó los comandos: `/ayuda` muestra los comandos disponibles y `/hola`, que no existe, devuelve el error «Comando desconocido» sin afectar la conexión.

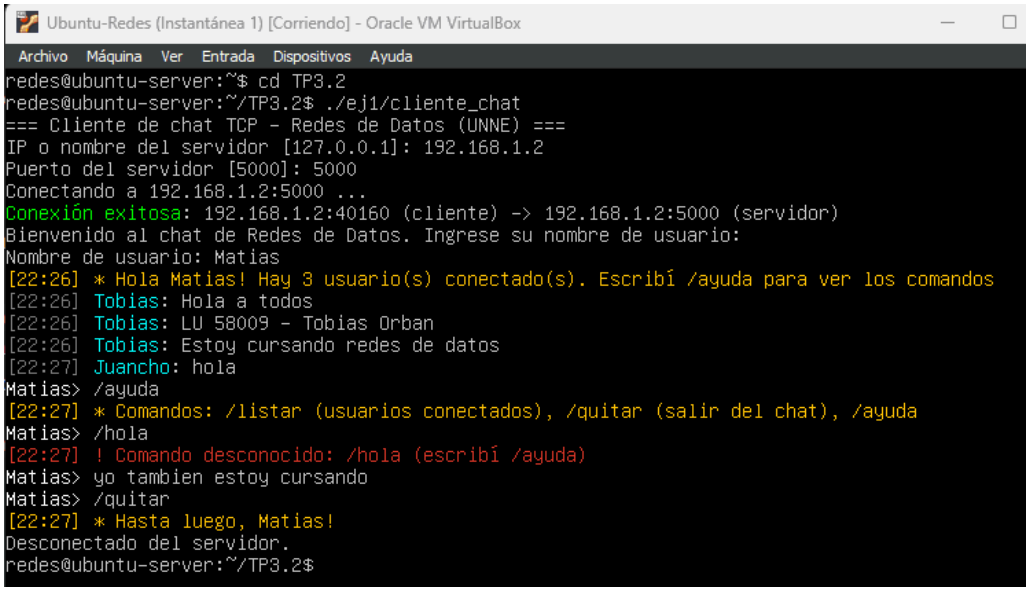


Figura 5. *Cliente «Matias»: recepción de mensajes, comando /ayuda, comando inexistente rechazado con un error y salida con /quitar.*

En una sesión de prueba anterior se verificó la caída del servidor: al detenerlo con Ctrl+C, el cliente recibió el aviso de apagado, detectó el cierre de la conexión y terminó informando la causa, sin quedar colgado.

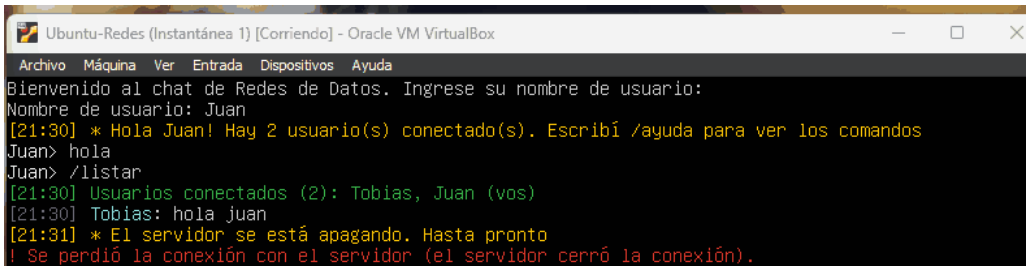


Figura 6. *Cliente «Juan» (sesión anterior): comando /listar, recepción de un mensaje y manejo de la caída del servidor.*

3. Ejercicio 2 — Servidor de consultas de HW/SW para inventario

Se implementó el **escenario 2**: un servidor que consulta características de hardware y software de los equipos clientes para armar un inventario, reutilizando la base del Ejercicio 1 (sockets TCP, protocolo por líneas, hilos y manejo de errores).

3.1. Diseño y roles

Siguiendo el esquema de la consigna, el **servidor pregunta** y el **cliente responde**. Sin embargo, la conexión TCP la inicia el cliente: el programa `agente_inventario`, que se ejecuta en cada equipo a inventariar, hace `connect()` hacia el servidor, que escucha en el puerto 6000. Así el servidor no necesita conocer de antemano las IP de los equipos, que se registran solos al iniciar el agente, y el esquema funciona aunque los equipos estén detrás de NAT. Una vez establecida la conexión, los roles de la capa de aplicación se invierten: el servidor envía consultas y el agente las contesta.

- **servidor_inventario.c** (575 líneas): un hilo acepta agentes en segundo plano mientras el hilo principal presenta un menú para listar los equipos, consultar uno, generar el inventario completo (guardado en un archivo de texto) o enviar una consulta a todos los equipos.
- **agente_inventario.c** (641 líneas): solicita IP y puerto del servidor, informa si la conexión fue exitosa, se presenta con el nombre del equipo y responde cada consulta leyendo `/proc`, `/sys`, `/etc` y la salida de comandos del sistema.

3.2. Protocolo de aplicación

Sentido	Mensaje	Significado
Agente → Servidor	`HOLA <hostname>`	Presentación al conectarse
Servidor → Agente	`OK <id>` / `ERR <motivo>`	Registro aceptado con un número de equipo, o rechazado
Servidor → Agente	`CONSULTA <tipo>`	Pedido de información (cpu, ram, disco, fs, …)
Agente → Servidor	`RESP <n>` + n bytes	Respuesta de n bytes de texto
Agente → Servidor	`ERR <n>` + n bytes	Consulta desconocida o mensaje no reconocido
Servidor → Agente	`FIN`	El servidor finaliza la sesión

A diferencia del chat, las respuestas ocupan muchas líneas, por lo que no alcanza con usar el salto de línea como delimitador. Cada respuesta va precedida de una cabecera con su **longitud en bytes**, y el servidor lee exactamente esa cantidad con `leer_bloque()` (líneas 145–162). Es la misma técnica que usa HTTP con el encabezado Content-Length. Ejemplo de intercambio:

```
agente   → servidor :  HOLA ubuntu-server
servidor → agente   :  OK 1
servidor → agente   :  CONSULTA cpu
agente   → servidor :  RESP 327
                       Fuente: /proc/cpuinfo
                         Modelo               ...     (327 bytes en total)
```

3.3. Consultas implementadas

Consulta	Información devuelta	Fuente de los datos
`cpu`	Modelo, fabricante, frecuencia, caché, núcleos, hilos, CPUs lógicas y arquitectura	`/proc/cpuinfo`, `sysconf()`, `uname()`
`ram`	Memoria total, libre, disponible, buffers y caché, porcentaje de uso y swap	`/proc/meminfo`, `free -h`
`disco`	Espacio de la partición raíz, discos físicos y tipo (HDD/SSD), uso por sistema de archivos	`statvfs()`, `lsblk -d`, `df -h`
`fs`	Dispositivos montados, punto de montaje, tipo y modo (lectura/escritura)	`/proc/mounts`, `df -hT`
`loadavg`	Carga promedio a 1, 5 y 15 minutos, procesos, carga por CPU y tiempo encendido	`/proc/loadavg`, `/proc/uptime`
`particiones`	Tabla de particiones con tamaños y sistemas de archivos	`/proc/partitions`, `lsblk`
`so`	Hostname, distribución, versión del kernel y arquitectura	`gethostname()`, `/etc/os-release`, `uname()`, `hostnamectl`
`red`	Interfaces con estado, MAC, MTU y direcciones IPv4/IPv6 con prefijo	`getifaddrs()`, `/sys/class/net`
`procesos`	Cantidad de procesos y los 20 con mayor consumo de CPU	`/proc`, `ps -eo … --sort=-%cpu`

Seguridad. El agente responde solo a esta lista cerrada de consultas (tabla `CONSULTAS`, líneas 410–413). Los comandos que ejecuta están fijos en el código y nunca se arma un comando con texto recibido por la red: de lo contrario, cualquiera que se conectara al agente obtendría una terminal remota sobre el equipo. El servidor, a su vez, filtra el hostname recibido y descarta toda conexión que no se presente con `HOLA`.

3.4. Comunicación por sockets: líneas relevantes

Servidor de inventario (servidor_inventario.c)

Línea	Instrucción	Función
505	`socket(AF_INET, SOCK_STREAM, 0)`	Socket TCP/IPv4 de escucha.
511	`setsockopt(SO_REUSEADDR)`	Reutilización inmediata del puerto.
516–519	`bind()`	Enlace a 0.0.0.0:6000.
529	`listen()`	Modo pasivo con cola de 10 conexiones.
536, 540	`pthread_create()`	Lanza el hilo aceptador, independiente del menú.
265	`accept()`	Acepta la conexión de un agente.
286, 325	`setsockopt(SO_RCVTIMEO)`	Tiempo máximo de 5 s para recibir HOLA y luego de 15 s para cada respuesta.
291	validación de `HOLA`	Rechaza conexiones que no son de un agente.
229	`send()`	Envía `CONSULTA <tipo>` (a través de `enviar_fmt()` y `enviar_todo()`).
231–248	`recv()`	Lee la cabecera `RESP <n>` y luego exactamente n bytes.
203	`recv(MSG_PEEK | MSG_DONTWAIT)`	Comprueba, sin consumir datos ni bloquearse, si un agente inactivo cerró la conexión.
189, 342	`close()`	Cierre de la conexión de un equipo.

Agente de inventario (agente_inventario.c)

Línea	Instrucción	Función
480	`getaddrinfo()`	Resuelve la IP ingresada por el usuario.
487	`socket()`	Socket TCP del agente.
494–506	`connect()` + `select()`	Conexión con tiempo máximo de 5 s, igual que en el cliente de chat.
582	`getsockname()`	Muestra «Conexión exitosa» con IP y puerto local y remoto.
590	`send()`	Presentación `HOLA <hostname>`.
605	`recv()`	Bucle que espera consultas del servidor.
466–471	`send()`	Envía `RESP <n>` seguido de los n bytes del informe.
608, 639	`close()`	Cierre ante `FIN` o pérdida de conexión.

3.5. Concurrencia y manejo de errores

- **Hilo aceptador + menú.** Los agentes pueden conectarse en cualquier momento, incluso mientras el operador usa el menú. La lista de equipos se protege con `mutex_equipos` y cada nuevo equipo se anuncia en pantalla con su número, hostname, IP y puerto.
- **Agente caído.** Si un agente se desconecta durante una consulta, `recv()` devuelve 0 o error y el equipo se quita de la lista con un aviso. Si se desconecta mientras está inactivo, se detecta al listar los equipos mediante `recv()` con `MSG_PEEK | MSG_DONTWAIT`.
- **Tiempos máximos.** 5 s para la presentación y 15 s para cada respuesta: un agente que no responde no bloquea indefinidamente al servidor.
- **Datos inválidos.** Las conexiones que no se presentan con `HOLA` se rechazan (por ejemplo, un cliente de chat que se conecta al puerto equivocado). Las respuestas mayores a 8 MB se descartan y las opciones inválidas del menú se informan. El agente responde `ERR` ante una consulta desconocida.
- **Cierre ordenado.** La opción 0 envía `FIN` a todos los agentes, que terminan informando «El servidor finalizó la sesión». El puerto ocupado y los errores de conexión se tratan igual que en el Ejercicio 1.

3.6. Pruebas de funcionamiento

Se ejecutó el servidor de inventario en la máquina virtual y el agente desde otra sesión, ingresando como servidor 192.168.1.2:6000. El agente confirmó la conexión, quedó registrado como equipo #1 y respondió una consulta de CPU y luego el inventario completo (opción 3 del menú): las nueve consultas, con el tamaño de cada respuesta. Al cerrar el servidor con la opción 0, el agente recibió `FIN` y terminó en forma ordenada. Al reiniciar el servidor, el agente volvió a conectarse (equipo #2) y respondió una consulta de sistemas de archivos.

```
redes@ubuntu-server:~/TP3.2$ ./ej2/agente_inventario
=== Agente de inventario HW/SW - Redes de Datos (UNNE) ===
IP del servidor de inventario [127.0.0.1]: 192.168.1.2
Puerto [6000]: 6000
Conectando a 192.168.1.2:6000 ...
Conexión exitosa: 192.168.1.2:41976 (agente) -> 192.168.1.2:6000 (servidor)
[21:32:38] Registrado como equipo #1 (ubuntu-server). Esperando consultas... (Ctrl+C para salir)
[21:32:59] Consulta 'cpu' respondida (327 bytes)
[21:33:15] Consulta 'cpu' respondida (327 bytes)
[21:33:15] Consulta 'ram' respondida (480 bytes)
[21:33:15] Consulta 'disco' respondida (589 bytes)
[21:33:15] Consulta 'fs' respondida (850 bytes)
[21:33:15] Consulta 'loadavg' respondida (236 bytes)
[21:33:15] Consulta 'particiones' respondida (1164 bytes)
[21:33:16] Consulta 'so' respondida (606 bytes)
[21:33:16] Consulta 'red' respondida (259 bytes)
[21:33:16] Consulta 'procesos' respondida (1105 bytes)
[21:34:24] El servidor finalizó la sesión.
redes@ubuntu-server:~/TP3.2$ ./ej2/agente_inventario
=== Agente de inventario HW/SW - Redes de Datos (UNNE) ===
IP del servidor de inventario [127.0.0.1]: 192.168.1.2
Puerto [6000]: 6000
Conectando a 192.168.1.2:6000 ...
Conexión exitosa: 192.168.1.2:54204 (agente) -> 192.168.1.2:6000 (servidor)
[21:35:02] Registrado como equipo #2 (ubuntu-server). Esperando consultas... (Ctrl+C para salir)
[21:35:43] Consulta 'fs' respondida (850 bytes)
```

Figura 7. *Agente de inventario: conexión exitosa, registro en el servidor, respuesta a las consultas del inventario completo, fin de sesión ordenado y reconexión.*

La consulta «fs» devuelve los dispositivos montados de la máquina virtual: la raíz sobre un volumen lógico LVM (`/dev/mapper/ubuntu--vg-ubuntu--lv`, ext4, 9,8 GB con 66 % de uso), la partición `/boot` en `/dev/sda2` y los paquetes snap montados como squashfs de solo lectura. La consulta se envió a los dos equipos registrados en ese momento, por lo que la respuesta aparece una vez por cada uno.

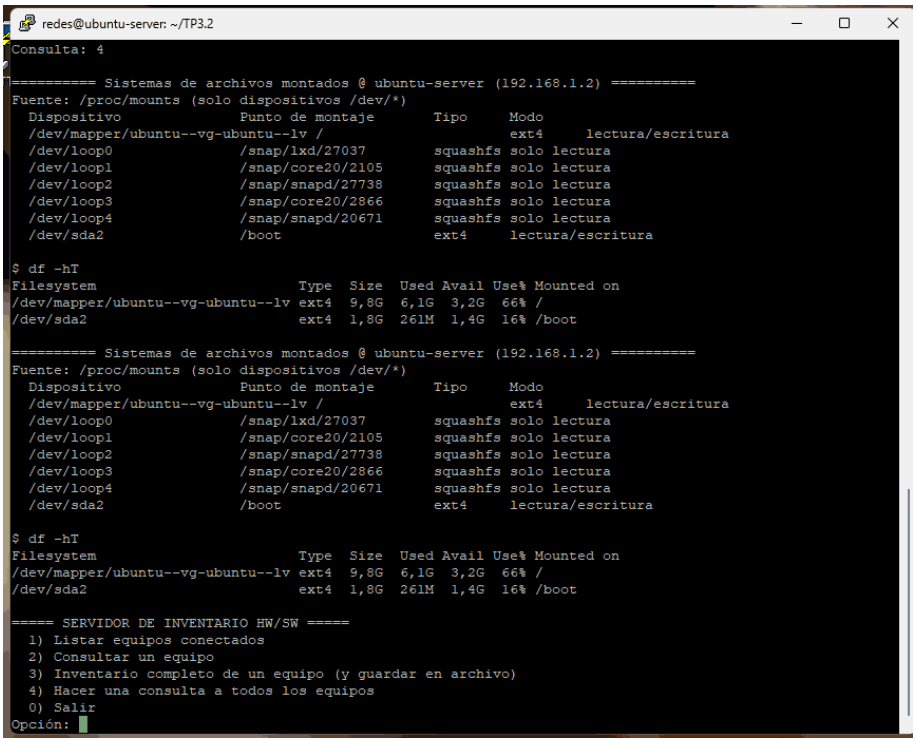


Figura 8. *Servidor de inventario: respuesta a la consulta «fs» (sistemas de archivos montados) de los equipos conectados y menú principal.*

3.7. Comparación con UDP

El servicio se implementó sobre TCP. La tabla resume qué implicaría hacerlo sobre UDP:

Aspecto	TCP (implementado)	UDP (alternativa)
Conexión	Se establece con el three-way handshake; el servidor sabe qué agentes están conectados y detecta su caída.	Sin conexión: haría falta que cada agente enviara mensajes periódicos de presencia para saber cuáles siguen activos.
Confiabilidad	Entrega garantizada, en orden y con retransmisión automática.	Los datagramas pueden perderse, duplicarse o llegar desordenados; la aplicación debería numerar, confirmar y reintentar.
Tamaño de las respuestas	Flujo de bytes: respuestas de cualquier tamaño (el inventario completo supera los 5 KB).	Cada respuesta debe entrar en un datagrama (máximo teórico 65 507 bytes, fragmentado por IP sobre una MTU de 1500); en la práctica habría que dividirla y reensamblarla.
Delimitación	Requiere marcar el fin de cada mensaje (longitud en la cabecera).	Conserva los límites de cada mensaje.
Costo	Handshake inicial, cabecera de 20 bytes y estado por conexión.	Menor latencia y cabecera de 8 bytes.

Para un inventario, donde importa que cada respuesta llegue completa y correcta más que la latencia, TCP es la opción adecuada. UDP conviene a mensajes breves e independientes, como las consultas DNS analizadas en el TP 3.1.

4. Ejercicio 3 — Inspección en Wireshark

Se capturó el tráfico del chat durante la sesión de prueba con tres clientes de la sección 2.9: las tres conexiones, el intercambio de mensajes y comandos, y la salida de cada usuario. La captura, guardada en `chat.pcap`, contiene 105 paquetes registrados durante 138,1 segundos.

4.1. Condiciones de la captura

La captura se realizó en la máquina virtual con `tcpdump`, la herramienta de captura por línea de comandos que usa la misma biblioteca (libpcap) y el mismo formato de archivo que Wireshark:

```
sudo tcpdump -i any -w chat.pcap port 5000
```

- `-i any`: captura en todas las interfaces a la vez, incluida la de loopback.
- `-w chat.pcap`: guarda los paquetes completos en el archivo, en lugar de mostrarlos por pantalla.
- `port 5000`: filtro de captura (sintaxis BPF); solo se registran los paquetes con origen o destino en el puerto del chat.

Luego el archivo se copió al equipo anfitrión y se abrió con Wireshark para su análisis.

Parámetro	Valor
Archivo	`chat.pcap`
Encapsulado	Linux cooked capture v2 (SLL2), propio de la pseudo-interfaz `any`
Interfaz real del tráfico	`lo` (loopback, índice de interfaz 1)
Filtro de captura	`port 5000`
Paquetes / duración	105 paquetes en 138,1 s
Servidor	192.168.1.2, puerto 5000/TCP

Servidor y clientes se ejecutaron en la misma máquina virtual y se conectaron a 192.168.1.2, la dirección de su propia interfaz `enp0s3`. Como el destino es una dirección local, el núcleo no envía esos paquetes a la red sino que los entrega a través de la interfaz de loopback: por eso la captura se hizo sobre `any`, ya que sobre `enp0s3` no habría registrado ningún paquete. La consecuencia directa es que **las dos puntas de cada conexión tienen la misma IP**, y los clientes se distinguen únicamente por su puerto de origen.

4.2. Identificación de clientes y tuplas de cada conexión

El menú *Statistics* → *Conversations* → *TCP* lista 3 conversaciones, una por cliente. Wireshark asigna a cada conexión un índice de flujo (`tcp.stream`). Para identificar a qué usuario corresponde cada una, se examina el primer segmento con datos enviado por el cliente, que transporta el nombre de usuario:

Cliente	Flujo (tcp.stream)	IP origen	Puerto origen	IP destino	Puerto destino	Paquetes	Bytes de datos	Paquete con el nombre
Tobias	0	192.168.1.2	38674	192.168.1.2	5000	36	443	6
Juancho	1	192.168.1.2	40280	192.168.1.2	5000	35	480	17
Matias	2	192.168.1.2	40160	192.168.1.2	5000	34	495	30

La tabla expresa cada conexión en el sentido cliente → servidor; en los segmentos que envía el servidor, origen y destino se invierten (192.168.1.2:5000 → 192.168.1.2:38674, por ejemplo). El puerto destino es

siempre 5000, el puerto fijo en el que escucha el servidor, mientras que los puertos de origen 38674, 40280 y 40160 son **efímeros**: los asigna el núcleo en el momento del `connect()`, dentro del rango que Linux reserva para ese fin (32768–60999 por defecto). Como los tres clientes comparten la IP, es el puerto de origen el que permite al núcleo entregar cada segmento al socket correcto.

Las tuplas coinciden con las que registró el servidor al aceptar cada conexión (Figura 2) y con las que informó cada cliente en su mensaje «Conexión exitosa» (Figuras 3, 4 y 5). Filtros de visualización utilizados:

```
tcp.port == 5000          todo el tráfico del chat
tcp.stream eq 0           conexión de Tobias
tcp.stream eq 1           conexión de Juancho
tcp.stream eq 2           conexión de Matias
tcp.flags.syn == 1        inicios de conexión (SYN y SYN-ACK)
tcp.flags.fin == 1        cierres de conexión
tcp.len > 0               solo segmentos que transportan datos
```

La lista completa de paquetes, en el formato de columnas del panel principal de Wireshark (con números de secuencia relativos y ventana escalada), es la siguiente:

No.	Time	Source	Destination	Len	Info
1	0.000	192.168.1.2	192.168.1.2	80	38674 → 5000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 WS=128
2	0.000	192.168.1.2	192.168.1.2	80	5000 → 38674 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 WS=128
3	0.000	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=1 Ack=1 Win=65536 Len=0
4	0.000	192.168.1.2	192.168.1.2	144	5000 → 38674 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=72
5	0.000	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=1 Ack=73 Win=65536 Len=0
6	2.850	192.168.1.2	192.168.1.2	79	38674 → 5000 [PSH, ACK] Seq=1 Ack=73 Win=65536 Len=7
7	2.850	192.168.1.2	192.168.1.2	72	5000 → 38674 [ACK] Seq=73 Ack=8 Win=65536 Len=0
8	2.850	192.168.1.2	192.168.1.2	82	5000 → 38674 [PSH, ACK] Seq=73 Ack=8 Win=65536 Len=10
9	2.850	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=8 Ack=83 Win=65536 Len=0
10	2.850	192.168.1.2	192.168.1.2	158	5000 → 38674 [PSH, ACK] Seq=83 Ack=8 Win=65536 Len=86
11	2.850	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=8 Ack=169 Win=65536 Len=0
12	36.960	192.168.1.2	192.168.1.2	80	40280 → 5000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 WS=128
13	36.960	192.168.1.2	192.168.1.2	80	5000 → 40280 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 WS=128
14	36.960	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=1 Ack=1 Win=65536 Len=0
15	36.961	192.168.1.2	192.168.1.2	144	5000 → 40280 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=72
16	36.961	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=1 Ack=73 Win=65536 Len=0
17	42.385	192.168.1.2	192.168.1.2	80	40280 → 5000 [PSH, ACK] Seq=1 Ack=73 Win=65536 Len=8
18	42.385	192.168.1.2	192.168.1.2	72	5000 → 40280 [ACK] Seq=73 Ack=9 Win=65536 Len=0
19	42.385	192.168.1.2	192.168.1.2	83	5000 → 40280 [PSH, ACK] Seq=73 Ack=9 Win=65536 Len=11
20	42.385	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=9 Ack=84 Win=65536 Len=0
21	42.385	192.168.1.2	192.168.1.2	159	5000 → 40280 [PSH, ACK] Seq=84 Ack=9 Win=65536 Len=87
22	42.385	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=9 Ack=171 Win=65536 Len=0
23	42.385	192.168.1.2	192.168.1.2	101	5000 → 38674 [PSH, ACK] Seq=169 Ack=8 Win=65536 Len=29
24	42.385	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=8 Ack=198 Win=65536 Len=0
25	68.628	192.168.1.2	192.168.1.2	80	40160 → 5000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 WS=128
26	68.628	192.168.1.2	192.168.1.2	80	5000 → 40160 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 WS=128
27	68.628	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=1 Ack=1 Win=65536 Len=0
28	68.628	192.168.1.2	192.168.1.2	144	5000 → 40160 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=72
29	68.628	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=1 Ack=73 Win=65536 Len=0
30	70.164	192.168.1.2	192.168.1.2	79	40160 → 5000 [PSH, ACK] Seq=1 Ack=73 Win=65536 Len=7
31	70.164	192.168.1.2	192.168.1.2	72	5000 → 40160 [ACK] Seq=73 Ack=8 Win=65536 Len=0
32	70.164	192.168.1.2	192.168.1.2	82	5000 → 40160 [PSH, ACK] Seq=73 Ack=8 Win=65536 Len=10
33	70.164	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=8 Ack=83 Win=65536 Len=0
34	70.164	192.168.1.2	192.168.1.2	158	5000 → 40160 [PSH, ACK] Seq=83 Ack=8 Win=65536 Len=86
35	70.164	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=8 Ack=169 Win=65536 Len=0
36	70.164	192.168.1.2	192.168.1.2	100	5000 → 38674 [PSH, ACK] Seq=198 Ack=8 Win=65536 Len=28
37	70.164	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=8 Ack=226 Win=65536 Len=0
38	70.164	192.168.1.2	192.168.1.2	100	5000 → 40280 [PSH, ACK] Seq=171 Ack=9 Win=65536 Len=28
39	70.164	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=9 Ack=199 Win=65536 Len=0
40	79.010	192.168.1.2	192.168.1.2	85	38674 → 5000 [PSH, ACK] Seq=8 Ack=226 Win=65536 Len=13
41	79.010	192.168.1.2	192.168.1.2	97	5000 → 40280 [PSH, ACK] Seq=199 Ack=9 Win=65536 Len=25
42	79.010	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=9 Ack=224 Win=65536 Len=0
43	79.010	192.168.1.2	192.168.1.2	97	5000 → 40160 [PSH, ACK] Seq=169 Ack=8 Win=65536 Len=25
44	79.010	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=8 Ack=194 Win=65536 Len=0
45	79.054	192.168.1.2	192.168.1.2	72	5000 → 38674 [ACK] Seq=226 Ack=21 Win=65536 Len=0
46	90.282	192.168.1.2	192.168.1.2	96	38674 → 5000 [PSH, ACK] Seq=21 Ack=226 Win=65536 Len=24
47	90.282	192.168.1.2	192.168.1.2	72	5000 → 38674 [ACK] Seq=226 Ack=45 Win=65536 Len=0
48	90.282	192.168.1.2	192.168.1.2	108	5000 → 40280 [PSH, ACK] Seq=224 Ack=9 Win=65536 Len=36
49	90.282	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=9 Ack=260 Win=65536 Len=0
50	90.282	192.168.1.2	192.168.1.2	108	5000 → 40160 [PSH, ACK] Seq=194 Ack=8 Win=65536 Len=36
51	90.282	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=8 Ack=230 Win=65536 Len=0
52	101.959	192.168.1.2	192.168.1.2	102	38674 → 5000 [PSH, ACK] Seq=45 Ack=226 Win=65536 Len=30
53	101.959	192.168.1.2	192.168.1.2	72	5000 → 38674 [ACK] Seq=226 Ack=75 Win=65536 Len=0
54	101.959	192.168.1.2	192.168.1.2	114	5000 → 40280 [PSH, ACK] Seq=260 Ack=9 Win=65536 Len=42
55	101.959	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=9 Ack=302 Win=65536 Len=0
56	101.959	192.168.1.2	192.168.1.2	114	5000 → 40160 [PSH, ACK] Seq=230 Ack=8 Win=65536 Len=42
57	101.959	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=8 Ack=272 Win=65536 Len=0
58	106.173	192.168.1.2	192.168.1.2	80	40280 → 5000 [PSH, ACK] Seq=9 Ack=302 Win=65536 Len=8
59	106.173	192.168.1.2	192.168.1.2	131	5000 → 40280 [PSH, ACK] Seq=302 Ack=17 Win=65536 Len=59
60	106.173	192.168.1.2	192.168.1.2	72	40280 → 5000 [ACK] Seq=17 Ack=361 Win=65536 Len=0
61	112.433	192.168.1.2	192.168.1.2	77	40280 → 5000 [PSH, ACK] Seq=17 Ack=361 Win=65536 Len=5
62	112.433	192.168.1.2	192.168.1.2	90	5000 → 38674 [PSH, ACK] Seq=226 Ack=75 Win=65536 Len=18
63	112.433	192.168.1.2	192.168.1.2	72	38674 → 5000 [ACK] Seq=75 Ack=244 Win=65536 Len=0
64	112.433	192.168.1.2	192.168.1.2	90	5000 → 40160 [PSH, ACK] Seq=272 Ack=8 Win=65536 Len=18
65	112.433	192.168.1.2	192.168.1.2	72	40160 → 5000 [ACK] Seq=8 Ack=290 Win=65536 Len=0

```
 66  112.474  192.168.1.2  192.168.1.2    72  5000 → 40280 [ACK] Seq=361 Ack=22 Win=65536 Len=0
 67  118.218  192.168.1.2  192.168.1.2    79  40160 → 5000 [PSH, ACK] Seq=8 Ack=290 Win=65536 Len=7
 68  118.218  192.168.1.2  192.168.1.2   150  5000 → 40160 [PSH, ACK] Seq=290 Ack=15 Win=65536 Len=78
 69  118.218  192.168.1.2  192.168.1.2    72  40160 → 5000 [ACK] Seq=15 Ack=368 Win=65536 Len=0
 70  121.800  192.168.1.2  192.168.1.2    78  40160 → 5000 [PSH, ACK] Seq=15 Ack=368 Win=65536 Len=6
 71  121.800  192.168.1.2  192.168.1.2   121  5000 → 40160 [PSH, ACK] Seq=368 Ack=21 Win=65536 Len=49
 72  121.800  192.168.1.2  192.168.1.2    72  40160 → 5000 [ACK] Seq=21 Ack=417 Win=65536 Len=0
 73  127.194  192.168.1.2  192.168.1.2    98  40160 → 5000 [PSH, ACK] Seq=21 Ack=417 Win=65536 Len=26
 74  127.194  192.168.1.2  192.168.1.2   110  5000 → 38674 [PSH, ACK] Seq=244 Ack=75 Win=65536 Len=38
 75  127.194  192.168.1.2  192.168.1.2    72  38674 → 5000 [ACK] Seq=75 Ack=282 Win=65536 Len=0
 76  127.194  192.168.1.2  192.168.1.2   110  5000 → 40280 [PSH, ACK] Seq=361 Ack=22 Win=65536 Len=38
 77  127.194  192.168.1.2  192.168.1.2    72  40280 → 5000 [ACK] Seq=22 Ack=399 Win=65536 Len=0
 78  127.238  192.168.1.2  192.168.1.2    72  5000 → 40160 [ACK] Seq=417 Ack=47 Win=65536 Len=0
 79  130.670  192.168.1.2  192.168.1.2    80  40160 → 5000 [PSH, ACK] Seq=47 Ack=417 Win=65536 Len=8
 80  130.670  192.168.1.2  192.168.1.2    72  5000 → 40160 [ACK] Seq=417 Ack=55 Win=65536 Len=0
 81  130.670  192.168.1.2  192.168.1.2    97  5000 → 40160 [PSH, ACK] Seq=417 Ack=55 Win=65536 Len=25
 82  130.670  192.168.1.2  192.168.1.2    72  40160 → 5000 [ACK] Seq=55 Ack=442 Win=65536 Len=0
 83  130.670  192.168.1.2  192.168.1.2    99  5000 → 38674 [PSH, ACK] Seq=282 Ack=75 Win=65536 Len=27
 84  130.670  192.168.1.2  192.168.1.2    72  38674 → 5000 [ACK] Seq=75 Ack=309 Win=65536 Len=0
 85  130.670  192.168.1.2  192.168.1.2    99  5000 → 40280 [PSH, ACK] Seq=399 Ack=22 Win=65536 Len=27
 86  130.670  192.168.1.2  192.168.1.2    72  40280 → 5000 [ACK] Seq=22 Ack=426 Win=65536 Len=0
 87  130.670  192.168.1.2  192.168.1.2    72  5000 → 40160 [FIN, ACK] Seq=442 Ack=55 Win=65536 Len=0
 88  130.671  192.168.1.2  192.168.1.2    72  40160 → 5000 [FIN, ACK] Seq=55 Ack=443 Win=65536 Len=0
 89  130.671  192.168.1.2  192.168.1.2    72  5000 → 40160 [ACK] Seq=443 Ack=56 Win=65536 Len=0
 90  134.813  192.168.1.2  192.168.1.2    80  40280 → 5000 [PSH, ACK] Seq=22 Ack=426 Win=65536 Len=8
 91  134.813  192.168.1.2  192.168.1.2    72  5000 → 40280 [ACK] Seq=426 Ack=30 Win=65536 Len=0
 92  134.814  192.168.1.2  192.168.1.2    98  5000 → 40280 [PSH, ACK] Seq=426 Ack=30 Win=65536 Len=26
 93  134.814  192.168.1.2  192.168.1.2    72  40280 → 5000 [ACK] Seq=30 Ack=452 Win=65536 Len=0
 94  134.814  192.168.1.2  192.168.1.2   100  5000 → 38674 [PSH, ACK] Seq=309 Ack=75 Win=65536 Len=28
 95  134.814  192.168.1.2  192.168.1.2    72  38674 → 5000 [ACK] Seq=75 Ack=337 Win=65536 Len=0
 96  134.814  192.168.1.2  192.168.1.2    72  5000 → 40280 [FIN, ACK] Seq=452 Ack=30 Win=65536 Len=0
 97  134.814  192.168.1.2  192.168.1.2    72  40280 → 5000 [FIN, ACK] Seq=30 Ack=453 Win=65536 Len=0
 98  134.814  192.168.1.2  192.168.1.2    72  5000 → 40280 [ACK] Seq=453 Ack=31 Win=65536 Len=0
 99  138.074  192.168.1.2  192.168.1.2    80  38674 → 5000 [PSH, ACK] Seq=75 Ack=337 Win=65536 Len=8
100  138.074  192.168.1.2  192.168.1.2    72  5000 → 38674 [ACK] Seq=337 Ack=83 Win=65536 Len=0
101  138.074  192.168.1.2  192.168.1.2    97  5000 → 38674 [PSH, ACK] Seq=337 Ack=83 Win=65536 Len=25
102  138.074  192.168.1.2  192.168.1.2    72  38674 → 5000 [ACK] Seq=83 Ack=362 Win=65536 Len=0
103  138.074  192.168.1.2  192.168.1.2    72  5000 → 38674 [FIN, ACK] Seq=362 Ack=83 Win=65536 Len=0
104  138.074  192.168.1.2  192.168.1.2    72  38674 → 5000 [FIN, ACK] Seq=83 Ack=363 Win=65536 Len=0
105  138.074  192.168.1.2  192.168.1.2    72  5000 → 38674 [ACK] Seq=363 Ack=84 Win=65536 Len=0
```

Figura 9. *Paquetes de chat.pcap: conexión de los tres clientes, registro de nombres, comandos, mensajes retransmitidos y cierre de las conexiones.*

Aislando la conexión de Tobias (flujo 0), Wireshark muestra solo los segmentos de la tupla 192.168.1.2:38674 ↔ 192.168.1.2:5000: el acuerdo de tres vías al comienzo, los segmentos de datos con sus confirmaciones y el cierre con FIN al final.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.2	192.168.1.2	TCP	80	38674 → 5000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1998976081 TSecr=0 WS=128
2	0.000024	192.168.1.2	192.168.1.2	TCP	80	5000 → 38674 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1998976081 TSecr=1998976081 WS=128
3	0.000034	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1998976081 TSecr=1998976081
4	0.000324	192.168.1.2	192.168.1.2	TCP	144	5000 → 38674 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=72 TSval=1998976081 TSecr=1998976081
5	0.000329	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=1 Ack=73 Win=65536 Len=0 TSval=1998976081 TSecr=1998976081
6	2.850249	192.168.1.2	192.168.1.2	TCP	79	38674 → 5000 [PSH, ACK] Seq=1 Ack=73 Win=65536 Len=7 TSval=1998978931 TSecr=1998976081
7	2.850264	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [ACK] Seq=73 Ack=8 Win=65536 Len=0 TSval=1998978931 TSecr=1998978931
8	2.850291	192.168.1.2	192.168.1.2	TCP	82	5000 → 38674 [PSH, ACK] Seq=73 Ack=8 Win=65536 Len=10 TSval=1998978931 TSecr=1998978931
9	2.850293	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=8 Ack=83 Win=65536 Len=0 TSval=1998978931 TSecr=1998978931
10	2.850300	192.168.1.2	192.168.1.2	TCP	158	5000 → 38674 [PSH, ACK] Seq=83 Ack=8 Win=65536 Len=86 TSval=1998978931 TSecr=1998978931
11	2.850303	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=8 Ack=169 Win=65536 Len=0 TSval=1998978931 TSecr=1998978931
23	42.385106	192.168.1.2	192.168.1.2	TCP	101	5000 → 38674 [PSH, ACK] Seq=169 Ack=8 Win=65536 Len=29 TSval=1999018466 TSecr=1998978931
24	42.385114	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=8 Ack=198 Win=65536 Len=0 TSval=1999018466 TSecr=1999018466
36	70.163866	192.168.1.2	192.168.1.2	TCP	100	5000 → 38674 [PSH, ACK] Seq=198 Ack=8 Win=65536 Len=28 TSval=1999046245 TSecr=1999018466
37	70.163869	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=8 Ack=226 Win=65536 Len=0 TSval=1999046245 TSecr=1999046245
40	79.009958	192.168.1.2	192.168.1.2	TCP	85	38674 → 5000 [PSH, ACK] Seq=8 Ack=226 Win=65536 Len=13 TSval=1999055091 TSecr=1999046245
45	79.053797	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [ACK] Seq=226 Ack=21 Win=65536 Len=0 TSval=1999055135 TSecr=1999055091
46	90.282011	192.168.1.2	192.168.1.2	TCP	96	38674 → 5000 [PSH, ACK] Seq=21 Ack=226 Win=65536 Len=24 TSval=1999066363 TSecr=1999055135
47	90.282023	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [ACK] Seq=226 Ack=45 Win=65536 Len=0 TSval=1999066363 TSecr=1999066363
52	101.959009	192.168.1.2	192.168.1.2	TCP	102	38674 → 5000 [PSH, ACK] Seq=45 Ack=226 Win=65536 Len=30 TSval=1999078040 TSecr=1999066363
53	101.959020	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [ACK] Seq=226 Ack=75 Win=65536 Len=0 TSval=1999078040 TSecr=1999078040
62	112.433179	192.168.1.2	192.168.1.2	TCP	90	5000 → 38674 [PSH, ACK] Seq=226 Ack=75 Win=65536 Len=18 TSval=1999088514 TSecr=1999078040
63	112.433186	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=75 Ack=244 Win=65536 Len=0 TSval=1999088514 TSecr=1999088514
74	127.193764	192.168.1.2	192.168.1.2	TCP	110	5000 → 38674 [PSH, ACK] Seq=244 Ack=75 Win=65536 Len=38 TSval=1999103275 TSecr=1999088514
75	127.193767	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=75 Ack=282 Win=65536 Len=0 TSval=1999103275 TSecr=1999103275
83	130.670176	192.168.1.2	192.168.1.2	TCP	99	5000 → 38674 [PSH, ACK] Seq=282 Ack=75 Win=65536 Len=27 TSval=1999106751 TSecr=1999103275
84	130.670178	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=75 Ack=309 Win=65536 Len=0 TSval=1999106751 TSecr=1999106751
94	134.813545	192.168.1.2	192.168.1.2	TCP	100	5000 → 38674 [PSH, ACK] Seq=309 Ack=75 Win=65536 Len=28 TSval=1999110894 TSecr=1999106751
95	134.813548	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=75 Ack=337 Win=65536 Len=0 TSval=1999110894 TSecr=1999110894
99	138.073859	192.168.1.2	192.168.1.2	TCP	80	38674 → 5000 [PSH, ACK] Seq=75 Ack=337 Win=65536 Len=8 TSval=1999114155 TSecr=1999110894
100	138.073878	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [ACK] Seq=337 Ack=83 Win=65536 Len=0 TSval=1999114155 TSecr=1999114155
101	138.073908	192.168.1.2	192.168.1.2	TCP	97	5000 → 38674 [PSH, ACK] Seq=337 Ack=83 Win=65536 Len=25 TSval=1999114155 TSecr=1999114155
102	138.073910	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [ACK] Seq=83 Ack=362 Win=65536 Len=0 TSval=1999114155 TSecr=1999114155
103	138.073941	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [FIN, ACK] Seq=362 Ack=83 Win=65536 Len=0 TSval=1999114155 TSecr=1999114155
104	138.074084	192.168.1.2	192.168.1.2	TCP	72	38674 → 5000 [FIN, ACK] Seq=83 Ack=363 Win=65536 Len=0 TSval=1999114155 TSecr=1999114155
105	138.074088	192.168.1.2	192.168.1.2	TCP	72	5000 → 38674 [ACK] Seq=363 Ack=84 Win=65536 Len=0 TSval=1999114155 TSecr=1999114155

Figura 10. *Wireshark: paquetes de la conexión de Tobias (flujo 0), desde el SYN inicial hasta el cierre con FIN.*

4.3. Establecimiento de las conexiones

Cada conexión comienza con el **acuerdo de tres vías** de TCP, que en el código corresponde al `connect()` del cliente (línea 244) y al `accept()` del servidor (línea 452):

Cliente	Paquetes	Instante	SYN	SYN-ACK	ACK
Tobias	1, 2, 3	0,0 s	:38674 → :5000 [SYN]	:5000 → :38674 [SYN, ACK]	:38674 → :5000 [ACK]
Juancho	12, 13, 14	37,0 s	:40280 → :5000 [SYN]	:5000 → :40280 [SYN, ACK]	:40280 → :5000 [ACK]
Matias	25, 26, 27	68,6 s	:40160 → :5000 [SYN]	:5000 → :40160 [SYN, ACK]	:40160 → :5000 [ACK]

- **SYN**: el cliente propone su número de secuencia inicial (Seq=0 relativo) y sus opciones: tamaño máximo de segmento MSS = 65495 bytes (la MTU de loopback, 65536, menos 40 bytes de cabeceras IP y TCP), SACK, marcas de tiempo y escala de ventana 7 (×128).
- **SYN-ACK**: el servidor confirma (Ack=1) y propone su propia secuencia inicial.
- **ACK**: el cliente confirma; a partir de aquí la ventana se informa escalada: 512 × 128 = 65 536 bytes.

El acuerdo completo demora menos de una décima de milisegundo, ya que no sale del equipo. Inmediatamente después, el servidor envía el primer mensaje de la aplicación, la bienvenida SYS (paquetes 4, 15 y 28): en este protocolo el servidor habla primero.

4.4. Flujo de datos no cifrado

Los datos viajan en texto plano: el panel de bytes muestra el contenido de cada segmento legible y la opción *Follow → TCP Stream* reconstruye la conversación completa de cada cliente. Ese mismo contenido, extraído de la captura, es el siguiente (C: cliente, S: servidor; entre corchetes, el número de paquete):

Flujo 0 — Tobias (192.168.1.2:38674 ↔ 192.168.1.2:5000)

```
[ 4] S → C  SYS Bienvenido al chat de Redes de Datos. Ingrese su nombre de usuario:
[ 6] C → S  Tobias
[ 8] S → C  OK Tobias
[10] S → C  SYS Hola Tobias! Hay 1 usuario(s) conectado(s). Escribí /ayuda para ver los comandos
[23] S → C  SYS Juancho se unió al chat
[36] S → C  SYS Matias se unió al chat
[40] C → S  Hola a todos
[46] C → S  LU 58009 - Tobias Orban
[52] C → S  Estoy cursando redes de datos
[62] S → C  MSG Juancho: hola
[74] S → C  MSG Matias: yo tambien estoy cursando
[83] S → C  SYS Matias salió del chat
[94] S → C  SYS Juancho salió del chat
[99] C → S  /quitar
[101] S → C  SYS Hasta luego, Tobias!
```

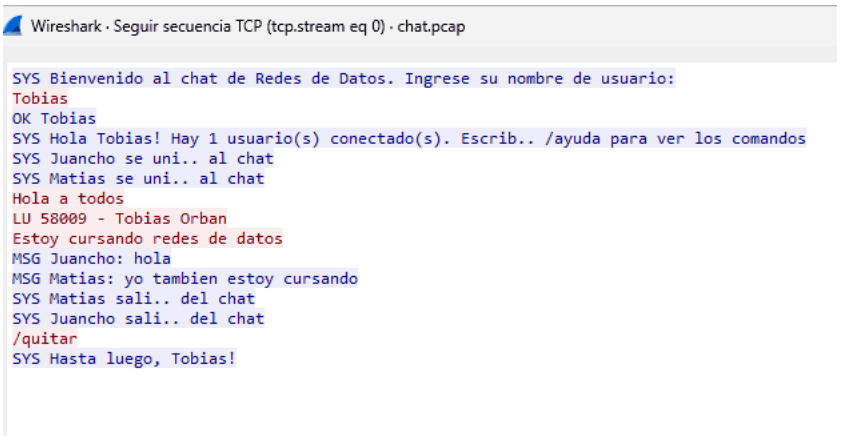


Figura 11. *Wireshark, Follow → TCP Stream del flujo 0: en rojo lo enviado por el cliente, en azul lo enviado por el servidor.*

Flujo 1 — Juancho (192.168.1.2:40280 ↔ 192.168.1.2:5000)

```
[15] S → C  SYS Bienvenido al chat de Redes de Datos. Ingrese su nombre de usuario:
[17] C → S  Juancho
[19] S → C  OK Juancho
[21] S → C  SYS Hola Juancho! Hay 2 usuario(s) conectado(s). Escribí /ayuda para ver los comandos
[38] S → C  SYS Matias se unió al chat
[41] S → C  MSG Tobias: Hola a todos
[48] S → C  MSG Tobias: LU 58009 - Tobias Orban
[54] S → C  MSG Tobias: Estoy cursando redes de datos
[58] C → S  /listar
[59] S → C  LST Usuarios conectados (3): Tobias, Juancho (vos), Matias
[61] C → S  hola
[76] S → C  MSG Matias: yo tambien estoy cursando
[85] S → C  SYS Matias salió del chat
[90] C → S  /quitar
[92] S → C  SYS Hasta luego, Juancho!
```

Flujo 2 — Matias (192.168.1.2:40160 ↔ 192.168.1.2:5000)

```
[28] S → C  SYS Bienvenido al chat de Redes de Datos. Ingrese su nombre de usuario:
[30] C → S  Matias
[32] S → C  OK Matias
[34] S → C  SYS Hola Matias! Hay 3 usuario(s) conectado(s). Escribí /ayuda para ver los comandos
[43] S → C  MSG Tobias: Hola a todos
[50] S → C  MSG Tobias: LU 58009 - Tobias Orban
[56] S → C  MSG Tobias: Estoy cursando redes de datos
[64] S → C  MSG Juancho: hola
[67] C → S  /ayuda
[68] S → C  SYS Comandos: /listar (usuarios conectados), /quitar (salir del chat), /ayuda
[70] C → S  /hola
[71] S → C  ERR Comando desconocido: /hola (escribí /ayuda)
[73] C → S  yo tambien estoy cursando
[79] C → S  /quitar
[81] S → C  SYS Hasta luego, Matias!
```

Se observan los nombres de usuario, los comandos `/listar`, `/ayuda` y `/quitar` con sus respuestas, y los mensajes del chat. Incluso los caracteres acentuados son visibles: «Escribí» aparece codificado en UTF-8, con la «í» representada por los bytes `C3 AD`.

4.5. Retransmisión de los mensajes por el servidor

La captura permite ver el funcionamiento central del servidor: cada mensaje que llega por una conexión sale por la conexión de cada uno de los demás clientes, y nunca por la del emisor. Todos los reenvíos se producen en menos de un milisegundo:

Paquetes	Recibido	De	Retransmitido	Hacia
40 → 41	Hola a todos	Tobias	MSG Tobias: Hola a todos	Juancho
40 → 43	Hola a todos	Tobias	MSG Tobias: Hola a todos	Matias

Paquetes	Recibido	De	Retransmitido	Hacia
46 → 48	`LU 58009 - Tobias Orban`	Tobias	`MSG Tobias: LU 58009 - Tobias Orban`	Juancho
46 → 50	`LU 58009 - Tobias Orban`	Tobias	`MSG Tobias: LU 58009 - Tobias Orban`	Matias
52 → 54	`Estoy cursando redes de datos`	Tobias	`MSG Tobias: Estoy cursando redes de datos`	Juancho
52 → 56	`Estoy cursando redes de datos`	Tobias	`MSG Tobias: Estoy cursando redes de datos`	Matias
61 → 62	`hola`	Juancho	`MSG Juancho: hola`	Tobias
61 → 64	`hola`	Juancho	`MSG Juancho: hola`	Matias
73 → 74	`yo tambien estoy cursando`	Matias	`MSG Matias: yo tambien estoy cursando`	Tobias
73 → 76	`yo tambien estoy cursando`	Matias	`MSG Matias: yo tambien estoy cursando`	Juancho

Del mismo modo, el registro de Matias genera el aviso `SYS Matias se unió al chat`, enviado por la conexión de cada usuario que ya estaba en el chat (paquetes 36 y 38).

4.6. Mensaje con la LU

Como texto identificable, el cliente Tobias envió el mensaje «LU 58009 - Tobias Orban». El filtro `tcp contains "58009"` lo localiza en 3 paquetes. El original es el paquete 46, enviado a los 90,3 s de iniciada la captura desde 192.168.1.2:38674 hacia 192.168.1.2:5000 (flujo 0). Lleva el indicador PSH, que pide al receptor entregar los datos a la aplicación sin esperar más segmentos, y transporta 24 bytes de datos. En el panel de bytes de Wireshark el texto se lee directamente:

```
tcp contains "58009"

0000   4c 55 20 35 38 30 30 39 20 2d 20 54 6f 62 69 61   LU 58009 - Tobia
0010   73 20 4f 72 62 61 6e 0a                           s Orban.
```

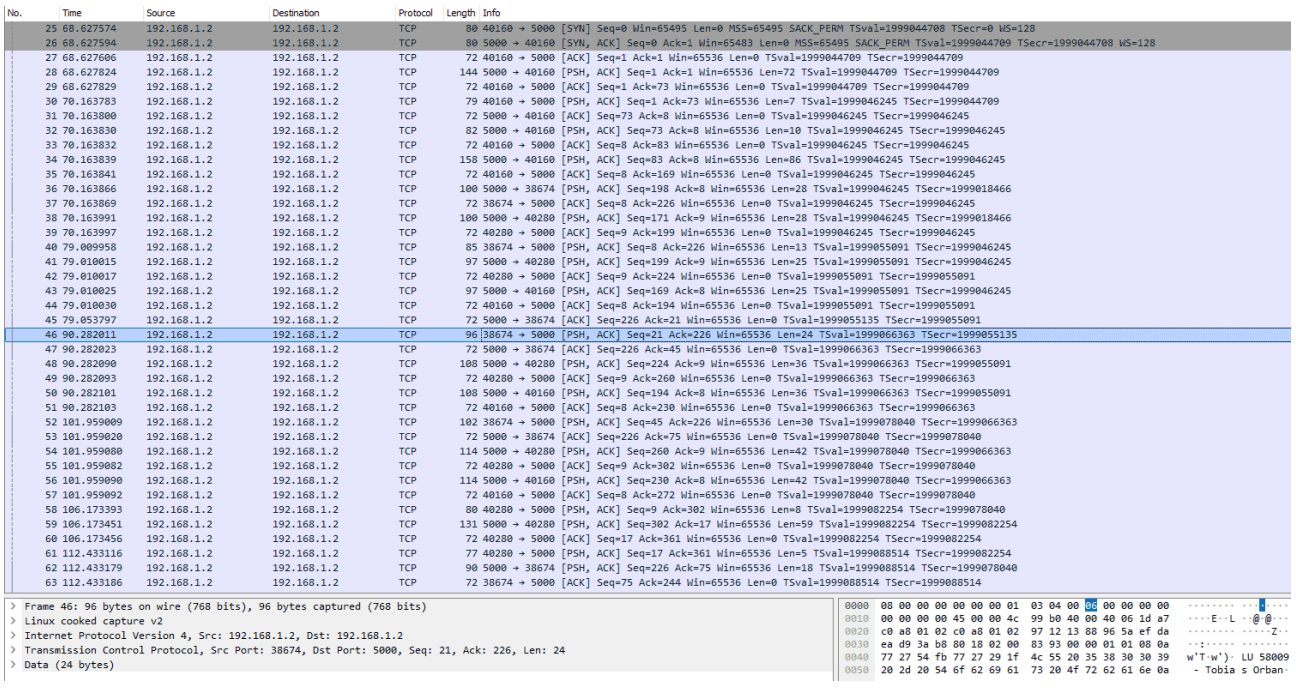


Figura 12. *Wireshark: paquete 46 seleccionado, con el mensaje que contiene la LU visible en el panel de bytes; debajo, los detalles de las capas Linux cooked capture, IPv4 y TCP.*

El mismo texto vuelve a aparecer, ahora con el prefijo `MSG`, en los paquetes 48 (hacia Juancho, 192.168.1.2:40280) y 50 (hacia Matias, 192.168.1.2:40160): son las copias que el servidor reenvió a los demás participantes.

Este es el comportamiento esperado de un protocolo sin cifrado, el mismo observado con HTTP en el TP 3.1: cualquier equipo que pueda observar el tráfico en algún punto del trayecto puede leer los nombres, los mensajes y los comandos, e incluso modificarlos. En una red real, la medida correspondiente sería cifrar la comunicación con TLS, por ejemplo con la biblioteca OpenSSL sobre los mismos sockets TCP, de modo que Wireshark solo mostraría registros cifrados.

4.7. Cierre de las conexiones

Cliente	/quitar	Despedida	FIN servidor	FIN cliente	ACK final	Instante
Matias	79	81	87	88	89	130,7 s
Juancho	90	92	96	97	98	134,8 s
Tobias	99	101	103	104	105	138,1 s

Los tres clientes salieron con `/quitar`, en el mismo orden que muestra el registro del servidor. En cada caso el servidor responde con la despedida, avisa a los demás que el usuario salió y luego cierra el socket (línea 357). Por eso el **primer FIN lo envía el servidor**, que realiza el cierre activo. El hilo receptor del cliente detecta en `recv()` el fin de la conexión y cierra su socket (línea 197), lo que genera su propio FIN, combinado con la confirmación del FIN recibido (FIN, ACK). El servidor confirma con un ACK final. El cierre resulta así de tres segmentos en lugar de cuatro.

Al cerrar primero, el socket del servidor queda en estado TIME_WAIT durante unos segundos, a la espera de segmentos atrasados. Esa es la situación en la que la opción `SO_REUSEADDR` del servidor permite reiniciarlo en el mismo puerto sin recibir el error «Address already in use».

5. Conclusiones

Se implementó en C un chat cliente-servidor sobre sockets TCP, con un servidor central concurrente que atiende a cada cliente en un hilo propio, retransmite los mensajes a los demás participantes, resuelve los comandos /listar y /quitar y tolera desconexiones normales y abruptas, la caída del propio servidor, el puerto ocupado y los datos inválidos. Sobre la misma base se desarrolló un servidor de inventario que consulta a los equipos su hardware y software a partir de /proc y de los comandos del sistema.

El desarrollo mostró que los sockets exponen TCP tal como es: un flujo de bytes sin límites entre mensajes. Esto obligó a definir en la capa de aplicación cómo delimitar cada mensaje: por saltos de línea en el chat y con una cabecera de longitud en el inventario.

La inspección con Wireshark permitió relacionar cada llamada del código con su efecto en la red. Cada `connect()` aparece como un acuerdo de tres vías, cada conexión queda identificada por su tupla IP:puerto, y en la captura se ve cómo el servidor recibe un mensaje por una conexión y lo reenvía por las demás. La prueba sobre loopback mostró además que, con todos los extremos en la misma IP, el puerto efímero de origen es lo que distingue a cada cliente. Por último, la lectura directa de los mensajes evidenció que TCP no protege la confidencialidad: esa función corresponde a una capa adicional como TLS.

Anexo A — Código fuente del Ejercicio 1

Se transcribe el código completo con su numeración de líneas, a la que hacen referencia las secciones anteriores. Los archivos fuente se adjuntan además a la entrega junto con el Makefile.

A.1. servidor_chat.c

```
/*
 * servidor_chat.c  -  TP 3.2 Redes de Datos (UNNE FaCENA) - Ejercicio 1
 *
 * Servidor central de chat sobre TCP. Acepta varios clientes en forma
 * concurrente (un hilo POSIX por cliente) y retransmite cada mensaje a
 * todos los demás clientes conectados.
 *
 * Compilar:  gcc -Wall -Wextra -pthread -o servidor_chat servidor_chat.c
 * Ejecutar:  ./servidor_chat [puerto]          (por defecto 5000)
 *
 * Protocolo de aplicación (texto plano, un mensaje por línea terminada en '\n'):
 *   Cliente -> Servidor
 *     1ra línea ........ nombre de usuario
 *     luego ............ texto libre, o los comandos /listar, /quitar, /ayuda
 *   Servidor -> Cliente (cada línea empieza con una etiqueta de 3 letras)
 *     OK  <nombre> ............ nombre aceptado
 *     MSG <usuario>: <texto> .. mensaje de otro usuario
 *     SYS <texto> ............. aviso del sistema (ingresos, salidas...)
 *     LST <texto> ............. respuesta a /listar
 *     ERR <texto> ............. error (nombre inválido, comando desconocido...)
 */
#define _GNU_SOURCE
#include <arpa/inet.h>      /* inet_ntop(), htons(), htonl()               */
#include <ctype.h>
#include <errno.h>
#include <ifaddrs.h>        /* getifaddrs(): listar IPs locales            */
#include <netinet/in.h>     /* struct sockaddr_in, INADDR_ANY              */
#include <pthread.h>        /* hilos: un hilo por cliente                  */
#include <signal.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <sys/socket.h>     /* socket(), bind(), listen(), accept(), send(), recv() */
#include <sys/time.h>
#include <time.h>
#include <unistd.h>         /* close()                                     */

#define PUERTO_DEFECTO   5000
#define MAX_CLIENTES     50
#define MAX_NOMBRE       20
#define MAX_LINEA        512
#define BACKLOG          10     /* conexiones pendientes en la cola de listen() */
#define INTENTOS_NOMBRE  3

typedef struct {
    char   datos[MAX_LINEA * 2];
    size_t len;
} lector_t;

typedef struct {
    int  fd;                        /* descriptor del socket conectado */
    char nombre[MAX_NOMBRE + 1];
    char ip[INET_ADDRSTRLEN];
    int  puerto;
} cliente_t;

static cliente_t *clientes[MAX_CLIENTES];          /* NULL = lugar libre */
static pthread_mutex_t mutex_clientes = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t mutex_log      = PTHREAD_MUTEX_INITIALIZER;
static volatile sig_atomic_t apagando = 0;
static int fd_escucha = -1;

/* ------------------------------------------------------------------ */
/* Utilidades                                                          */
/* ------------------------------------------------------------------ */

static void log_srv(const char *fmt, ...)
{
    char hora[16];
    time_t t = time(NULL);
    struct tm tm;
    localtime_r(&t, &tm);
    strftime(hora, sizeof hora, "%H:%M:%S", &tm);

    va_list ap;
```

```
    va_start(ap, fmt);
    pthread_mutex_lock(&mutex_log);
    printf("[%s] ", hora);
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    pthread_mutex_unlock(&mutex_log);
    va_end(ap);
}

/* send() puede enviar menos bytes de los pedidos: se repite hasta enviar todo.
 * MSG_NOSIGNAL evita que el proceso muera con SIGPIPE si el cliente ya cerró. */
static int enviar_todo(int fd, const char *buf, size_t len)
{
    while (len > 0) {
        ssize_t n = send(fd, buf, len, MSG_NOSIGNAL);           /* send() */
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        buf += n;
        len -= (size_t)n;
    }
    return 0;
}

/* Envía una línea con formato printf y agrega el '\n' del protocolo. */
static int enviar_fmt(int fd, const char *fmt, ...)
{
    char buf[MAX_LINEA + 64];
    va_list ap;
    va_start(ap, fmt);
    int n = vsnprintf(buf, sizeof buf - 1, fmt, ap);
    va_end(ap);
    if (n < 0) return -1;
    if ((size_t)n > sizeof buf - 2) n = (int)(sizeof buf - 2);
    buf[n++] = '\n';
    return enviar_todo(fd, buf, (size_t)n);
}

/* TCP es un flujo de bytes, no de mensajes: un recv() puede traer media línea
 * o varias juntas. Esta función acumula lo recibido hasta encontrar '\n'.
 * Retorna 1 = línea leída, 0 = el otro extremo cerró, -1 = error. */
static int leer_linea(int fd, lector_t *lr, char *linea, size_t tam)
{
    for (;;) {
        char  *nl = memchr(lr->datos, '\n', lr->len);
        size_t n;

        if (nl)
            n = (size_t)(nl - lr->datos);
        else if (lr->len == sizeof lr->datos)
            n = lr->len;                        /* línea demasiado larga: se corta */
        else {
            ssize_t r = recv(fd, lr->datos + lr->len,               /* recv() */
                             sizeof lr->datos - lr->len, 0);
            if (r == 0) return 0;               /* el cliente cerró la conexión */
            if (r < 0) {
                if (errno == EINTR) continue;
                return -1;                      /* ej.: ECONNRESET (corte abrupto) */
            }
            lr->len += (size_t)r;
            continue;
        }

        size_t copiar = n < tam - 1 ? n : tam - 1;
        memcpy(linea, lr->datos, copiar);
        linea[copiar] = '\0';
        size_t consumir = nl ? n + 1 : n;
        memmove(lr->datos, lr->datos + consumir, lr->len - consumir);
        lr->len -= consumir;
        if (copiar > 0 && linea[copiar - 1] == '\r')   /* clientes Windows/telnet */
            linea[copiar - 1] = '\0';
        return 1;
    }
}

/* Reemplaza caracteres de control (datos inválidos) por espacios. */
static void sanear(char *s)
{
    for (; *s; s++)
        if ((unsigned char)*s < 32 || *s == 127) *s = ' ';
}

static void recortar(char *s)
{
    size_t i = 0, n = strlen(s);
    while (i < n && isspace((unsigned char)s[i])) i++;
```

```
    while (n > i && isspace((unsigned char)s[n - 1])) n--;
    memmove(s, s + i, n - i);
    s[n - i] = '\0';
}

static const char *validar_nombre(const char *n)
{
    size_t len = strlen(n);
    if (len == 0) return "El nombre no puede estar vacío";
    if (len > MAX_NOMBRE) return "El nombre es demasiado largo (máximo 20 caracteres)";
    if (n[0] == '/') return "El nombre no puede empezar con '/'";
    for (size_t i = 0; i < len; i++) {
        unsigned char c = (unsigned char)n[i];
        if (!isalnum(c) && c != '_' && c != '-' && c != '.')
            return "Use solo letras, números, '_', '-' o '.' (sin espacios ni tildes)";
    }
    return NULL;
}

/* ------------------------------------------------------------------ */
/* Lista de clientes conectados (protegida con mutex)                  */
/* ------------------------------------------------------------------ */

/* 0 = registrado, -1 = nombre en uso, -2 = servidor lleno */
static int registrar(cliente_t *c, const char *nombre)
{
    int libre = -1, res = 0;
    pthread_mutex_lock(&mutex_clientes);
    for (int i = 0; i < MAX_CLIENTES; i++) {
        if (clientes[i] == NULL) {
            if (libre < 0) libre = i;
        } else if (strcasecmp(clientes[i]->nombre, nombre) == 0) {
            res = -1;
        }
    }
    if (res == 0 && libre < 0) res = -2;
    if (res == 0) {
        snprintf(c->nombre, sizeof c->nombre, "%s", nombre);
        clientes[libre] = c;
    }
    pthread_mutex_unlock(&mutex_clientes);
    return res;
}

static void quitar(cliente_t *c)
{
    pthread_mutex_lock(&mutex_clientes);
    for (int i = 0; i < MAX_CLIENTES; i++)
        if (clientes[i] == c) clientes[i] = NULL;
    pthread_mutex_unlock(&mutex_clientes);
}

static int cantidad_conectados(void)
{
    int n = 0;
    pthread_mutex_lock(&mutex_clientes);
    for (int i = 0; i < MAX_CLIENTES; i++)
        if (clientes[i]) n++;
    pthread_mutex_unlock(&mutex_clientes);
    return n;
}

/* Retransmite una línea a todos los clientes conectados excepto 'origen'. */
static void difundir(const cliente_t *origen, const char *fmt, ...)
{
    char buf[MAX_LINEA + 64];
    va_list ap;
    va_start(ap, fmt);
    int n = vsnprintf(buf, sizeof buf - 1, fmt, ap);
    va_end(ap);
    if (n < 0) return;
    if ((size_t)n > sizeof buf - 2) n = (int)(sizeof buf - 2);
    buf[n++] = '\n';

    pthread_mutex_lock(&mutex_clientes);
    for (int i = 0; i < MAX_CLIENTES; i++)
        if (clientes[i] && clientes[i] != origen)
            enviar_todo(clientes[i]->fd, buf, (size_t)n);   /* si falla, el hilo de
                                                               ese cliente lo detecta */
    pthread_mutex_unlock(&mutex_clientes);
}

static void enviar_lista(const cliente_t *c)
{
    char lista[MAX_LINEA];
    size_t usado = 0;
    int n = 0;
    lista[0] = '\0';
```

```
    pthread_mutex_lock(&mutex_clientes);
    for (int i = 0; i < MAX_CLIENTES; i++) {
        if (!clientes[i]) continue;
        int w = snprintf(lista + usado, sizeof lista - usado, "%s%s%s",
                         n ? ", " : "", clientes[i]->nombre,
                         clientes[i] == c ? " (vos)" : "");
        if (w > 0 && (size_t)w < sizeof lista - usado) usado += (size_t)w;
        n++;
    }
    pthread_mutex_unlock(&mutex_clientes);
    enviar_fmt(c->fd, "LST Usuarios conectados (%d): %s", n, lista);
}

/* ------------------------------------------------------------------ */
/* Hilo que atiende a un cliente                                      */
/* ------------------------------------------------------------------ */

static void *atender_cliente(void *arg)
{
    cliente_t *c = arg;
    lector_t   lr = { .len = 0 };
    char       linea[MAX_LINEA];
    int        r, registrado = 0;

    /* 1) Pedir y validar el nombre de usuario */
    enviar_fmt(c->fd, "SYS Bienvenido al chat de Redes de Datos. Ingrese su nombre de usuario:");
    for (int intento = 1; intento <= INTENTOS_NOMBRE && !registrado; intento++) {
        r = leer_linea(c->fd, &lr, linea, sizeof linea);
        if (r <= 0) {
            log_srv("%s:%d se desconectó antes de elegir un nombre", c->ip, c->puerto);
            goto fin;
        }
        sanear(linea);
        recortar(linea);
        const char *error = validar_nombre(linea);
        if (error) {
            enviar_fmt(c->fd, "ERR %s", error);
            continue;
        }
        int res = registrar(c, linea);
        if (res == -1) {
            enviar_fmt(c->fd, "ERR El nombre '%s' ya está en uso, elija otro", linea);
        } else if (res == -2) {
            enviar_fmt(c->fd, "ERR Servidor lleno (máximo %d usuarios). Intente más tarde", MAX_CLIENTES);
            log_srv("%s:%d rechazado: servidor lleno", c->ip, c->puerto);
            goto fin;
        } else {
            registrado = 1;
        }
    }
    if (!registrado) {
        enviar_fmt(c->fd, "ERR Demasiados intentos fallidos. Conexión cerrada");
        log_srv("%s:%d rechazado: nombre inválido %d veces", c->ip, c->puerto, INTENTOS_NOMBRE);
        goto fin;
    }

    enviar_fmt(c->fd, "OK %s", c->nombre);
    enviar_fmt(c->fd, "SYS Hola %s! Hay %d usuario(s) conectado(s). Escribí /ayuda para ver los comandos",
               c->nombre, cantidad_conectados());
    log_srv("%s:%d se registró como '%s' (conectados: %d)",
            c->ip, c->puerto, c->nombre, cantidad_conectados());
    difundir(c, "SYS %s se unió al chat", c->nombre);

    /* 2) Bucle principal: recibir líneas y retransmitirlas */
    int salida_normal = 0;
    while ((r = leer_linea(c->fd, &lr, linea, sizeof linea)) == 1) {
        sanear(linea);
        recortar(linea);
        if (linea[0] == '\0') continue;                /* ignorar líneas vacías */

        if (linea[0] == '/') {                         /* comandos especiales */
            if (strcasecmp(linea, "/listar") == 0) {
                enviar_lista(c);
            } else if (strcasecmp(linea, "/quitar") == 0) {
                enviar_fmt(c->fd, "SYS Hasta luego, %s!", c->nombre);
                salida_normal = 1;
                break;
            } else if (strcasecmp(linea, "/ayuda") == 0) {
                enviar_fmt(c->fd, "SYS Comandos: /listar (usuarios conectados), /quitar (salir del chat), /ayuda");
            } else {
                enviar_fmt(c->fd, "ERR Comando desconocido: %s (escribí /ayuda)", linea);
            }
            continue;
        }

        log_srv("%s: %s", c->nombre, linea);
        difundir(c, "MSG %s: %s", c->nombre, linea);    /* a todos menos al emisor */
    }
```

```
342
343     /* 3) Desconexión: normal (/quitar) o abrupta (cierre sin /quitar, RST...) */
344     quitar(c);
345     if (apagando) {
346         /* el servidor se está apagando: no avisar uno por uno */
347     } else if (salida_normal) {
348         log_srv("'%s' (%s:%d) salió con /quitar", c->nombre, c->ip, c->puerto);
349         difundir(NULL, "SYS %s salió del chat", c->nombre);
350     } else {
351         log_srv("'%s' (%s:%d) se desconectó abruptamente (%s)", c->nombre, c->ip, c->puerto,
352                 r == 0 ? "cerró sin /quitar" : strerror(errno));
353         difundir(NULL, "SYS %s se desconectó inesperadamente", c->nombre);
354     }
355
356 fin:
357     close(c->fd);                                                   /* close() */
358     free(c);
359     return NULL;
360 }
361
362 /* ---------------------------------------------------------------------- */
363
364 static void al_recibir_senal(int sig)
365 {
366     (void)sig;
367     apagando = 1;
368     if (fd_escucha >= 0)
369         shutdown(fd_escucha, SHUT_RDWR);   /* despierta al accept() bloqueado */
370 }
371
372 static void mostrar_direcciones(int puerto)
373 {
374     struct ifaddrs *ifs, *i;
375     log_srv("Servidor escuchando en el puerto TCP %d. Direcciones de este equipo:", puerto);
376     if (getifaddrs(&ifs) == 0) {
377         for (i = ifs; i; i = i->ifa_next) {
378             if (!i->ifa_addr || i->ifa_addr->sa_family != AF_INET) continue;
379             char ip[INET_ADDRSTRLEN];
380             inet_ntop(AF_INET, &((struct sockaddr_in *)i->ifa_addr)->sin_addr, ip, sizeof ip);
381             log_srv("   %-10s %s:%d", i->ifa_name, ip, puerto);
382         }
383         freeifaddrs(ifs);
384     }
385     log_srv("Ctrl+C para apagar el servidor.");
386 }
387
388 int main(int argc, char *argv[])
389 {
390     int puerto = PUERTO_DEFECTO;
391     if (argc > 1) {
392         char *fin;
393         long p = strtol(argv[1], &fin, 10);
394         if (*fin != '\0' || p < 1 || p > 65535) {
395             fprintf(stderr, "Puerto inválido: '%s'. Uso: %s [puerto 1-65535]\n", argv[1], argv[0]);
396             return 1;
397         }
398         puerto = (int)p;
399     }
400
401     signal(SIGPIPE, SIG_IGN);                 /* no morir si un cliente se cae */
402     struct sigaction sa;
403     memset(&sa, 0, sizeof sa);
404     sa.sa_handler = al_recibir_senal;
405     sigemptyset(&sa.sa_mask);
406     sigaction(SIGINT, &sa, NULL);
407     sigaction(SIGTERM, &sa, NULL);
408
409     /* (1) socket(): socket IPv4 (AF_INET) de tipo flujo (SOCK_STREAM) = TCP */
410     fd_escucha = socket(AF_INET, SOCK_STREAM, 0);
411     if (fd_escucha < 0) {
412         perror("socket");
413         return 1;
414     }
415
416     /* Permite reutilizar el puerto enseguida tras reiniciar el servidor
417      * (evita "Address already in use" por conexiones en TIME_WAIT). */
418     int si = 1;
419     setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof si);
420
421     /* (2) bind(): asocia el socket a todas las interfaces y al puerto elegido */
422     struct sockaddr_in dir;
423     memset(&dir, 0, sizeof dir);
424     dir.sin_family      = AF_INET;
425     dir.sin_addr.s_addr = htonl(INADDR_ANY);  /* 0.0.0.0: cualquier IP local */
426     dir.sin_port        = htons((uint16_t)puerto);  /* orden de red (big endian) */
427     if (bind(fd_escucha, (struct sockaddr *)&dir, sizeof dir) < 0) {
428         if (errno == EADDRINUSE)
429             fprintf(stderr, "Error: el puerto %d ya está en uso (¿otro servidor abierto?).\n"
```

```
                            "Cierre el otro proceso o use otro puerto, por ejemplo: %s %d\n",
                    puerto, argv[0], puerto + 1);
        else if (errno == EACCES)
            fprintf(stderr, "Error: los puertos menores a 1024 requieren permisos de root.\n");
        else
            perror("bind");
        close(fd_escucha);
        return 1;
    }

    /* (3) listen(): el socket pasa a modo pasivo y espera conexiones */
    if (listen(fd_escucha, BACKLOG) < 0) {
        perror("listen");
        close(fd_escucha);
        return 1;
    }
    mostrar_direcciones(puerto);

    /* (4) accept(): bucle que acepta clientes y crea un hilo para cada uno */
    while (!apagando) {
        struct sockaddr_in cli;
        socklen_t lc = sizeof cli;
        int fd = accept(fd_escucha, (struct sockaddr *)&cli, &lc);
        if (fd < 0) {
            if (apagando) break;
            if (errno == EINTR || errno == ECONNABORTED) continue;
            log_srv("accept falló: %s (el servidor sigue funcionando)", strerror(errno));
            sleep(1);
            continue;
        }

        /* Si un cliente deja de leer, send() no bloquea al servidor más de 3 s */
        struct timeval tv = { .tv_sec = 3, .tv_usec = 0 };
        setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &tv, sizeof tv);

        cliente_t *c = calloc(1, sizeof *c);
        if (!c) {
            close(fd);
            continue;
        }
        c->fd = fd;
        inet_ntop(AF_INET, &cli.sin_addr, c->ip, sizeof c->ip);
        c->puerto = ntohs(cli.sin_port);
        log_srv("Nueva conexión desde %s:%d", c->ip, c->puerto);

        pthread_t hilo;
        if (pthread_create(&hilo, NULL, atender_cliente, c) != 0) {
            enviar_fmt(fd, "ERR El servidor no puede atender más conexiones");
            close(fd);
            free(c);
            continue;
        }
        pthread_detach(hilo);
    }

    /* Apagado ordenado: avisar a los clientes y cerrar sus conexiones */
    log_srv("Apagando el servidor...");
    pthread_mutex_lock(&mutex_clientes);
    for (int i = 0; i < MAX_CLIENTES; i++) {
        if (!clientes[i]) continue;
        enviar_fmt(clientes[i]->fd, "SYS El servidor se está apagando. Hasta pronto");
        shutdown(clientes[i]->fd, SHUT_RDWR);
    }
    pthread_mutex_unlock(&mutex_clientes);
    close(fd_escucha);
    usleep(300000);          /* dar tiempo a los hilos a cerrar sus sockets */
    log_srv("Servidor detenido.");
    return 0;
}
```

A.2. cliente_chat.c

```
/*
 * cliente_chat.c  -  TP 3.2 Redes de Datos (UNNE FaCENA) - Ejercicio 1
 *
 * Cliente de chat por consola sobre TCP. Usa dos hilos:
 *   - hilo principal: lee lo que escribe el usuario y lo envía al servidor
 *   - hilo receptor : recibe los mensajes del servidor y los muestra en tiempo
 *                     real (la recepción no se bloquea mientras se escribe)
 *
 * Compilar:  gcc -Wall -Wextra -pthread -o cliente_chat cliente_chat.c
 * Ejecutar:  ./cliente_chat [host] [puerto]   (si faltan, se piden por teclado)
 *
 * Comandos: /listar  /quitar  /ayuda    (Ctrl+C o Ctrl+D también salen con /quitar)
 */
#define _GNU_SOURCE
#include <arpa/inet.h>      /* inet_ntop(), ntohs()                          */
```

```c
#include <ctype.h>
#include <errno.h>
#include <fcntl.h>          /* fcntl(): socket no bloqueante para el timeout */
#include <netdb.h>          /* getaddrinfo(): resolver nombre/IP del servidor */
#include <pthread.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <sys/select.h>
#include <sys/socket.h>     /* socket(), connect(), send(), recv()           */
#include <time.h>
#include <unistd.h>

#define HOST_DEFECTO      "127.0.0.1"
#define PUERTO_DEFECTO    "5000"
#define MAX_LINEA         512
#define MAX_MENSAJE       400
#define TIMEOUT_CONEXION  5        /* segundos */

#define RESET    "\033[0m"
#define NEGRITA  "\033[1m"
#define CIAN     "\033[36m"
#define AMARILLO "\033[33m"
#define ROJO     "\033[31m"
#define VERDE    "\033[32m"
#define GRIS     "\033[90m"
#define COL(c)   (interactivo ? (c) : "")

typedef struct {
    char   datos[MAX_LINEA * 2];
    size_t len;
} lector_t;

static int      fd_srv = -1;
static lector_t lector;
static char     mi_nombre[32] = "";
static int      interactivo = 0;
static volatile sig_atomic_t saliendo = 0;
static pthread_mutex_t mutex_pantalla = PTHREAD_MUTEX_INITIALIZER;

/* ------------------------------------------------------------------- */

static int enviar_todo(int fd, const char *buf, size_t len)
{
    while (len > 0) {
        ssize_t n = send(fd, buf, len, MSG_NOSIGNAL);           /* send() */
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        buf += n;
        len -= (size_t)n;
    }
    return 0;
}

static int enviar_linea(int fd, const char *texto)
{
    char buf[MAX_LINEA + 2];
    int n = snprintf(buf, sizeof buf, "%s\n", texto);
    if (n < 0 || (size_t)n >= sizeof buf) return -1;
    return enviar_todo(fd, buf, (size_t)n);
}

/* Igual que en el servidor: junta bytes hasta encontrar '\n'.
 * 1 = línea, 0 = servidor cerró, -1 = error */
static int leer_linea(int fd, lector_t *lr, char *linea, size_t tam)
{
    for (;;) {
        char  *nl = memchr(lr->datos, '\n', lr->len);
        size_t n;
        if (nl)
            n = (size_t)(nl - lr->datos);
        else if (lr->len == sizeof lr->datos)
            n = lr->len;
        else {
            ssize_t r = recv(fd, lr->datos + lr->len,                /* recv() */
                             sizeof lr->datos - lr->len, 0);
            if (r == 0) return 0;
            if (r < 0) {
                if (errno == EINTR) continue;
                return -1;
            }
            lr->len += (size_t)r;
            continue;
        }
```

```
        size_t copiar = n < tam - 1 ? n : tam - 1;
        memcpy(linea, lr->datos, copiar);
        linea[copiar] = '\0';
        size_t consumir = nl ? n + 1 : n;
        memmove(lr->datos, lr->datos + consumir, lr->len - consumir);
        lr->len -= consumir;
        if (copiar > 0 && linea[copiar - 1] == '\r') linea[copiar - 1] = '\0';
        return 1;
    }
}

static void recortar(char *s)
{
    size_t i = 0, n = strlen(s);
    while (i < n && isspace((unsigned char)s[i])) i++;
    while (n > i && isspace((unsigned char)s[n - 1])) n--;
    memmove(s, s + i, n - i);
    s[n - i] = '\0';
}

/* Pide un dato por teclado; si se deja vacío usa 'defecto' (si no es NULL). */
static void pedir(const char *mensaje, char *dst, size_t tam, const char *defecto)
{
    printf("%s", mensaje);
    fflush(stdout);
    if (!fgets(dst, (int)tam, stdin)) {
        printf("\n");
        exit(0);
    }
    dst[strcspn(dst, "\r\n")] = '\0';
    recortar(dst);
    if (dst[0] == '\0' && defecto) snprintf(dst, tam, "%s", defecto);
}

/* Debe llamarse con mutex_pantalla tomado */
static void mostrar_prompt(void)
{
    if (!interactivo) return;
    printf("%s%s>%s ", COL(NEGRITA), mi_nombre, COL(RESET));
    fflush(stdout);
}

static void hora_actual(char *dst, size_t tam)
{
    time_t t = time(NULL);
    struct tm tm;
    localtime_r(&t, &tm);
    strftime(dst, tam, "%H:%M", &tm);
}

/* Muestra una línea recibida del servidor con un color según su etiqueta */
static void mostrar_del_servidor(const char *linea)
{
    char h[8];
    hora_actual(h, sizeof h);
    pthread_mutex_lock(&mutex_pantalla);
    if (interactivo) printf("\r\033[K");      /* borra el prompt de la línea actual */

    if (strncmp(linea, "MSG ", 4) == 0) {
        const char *txt = linea + 4;
        const char *sep = strstr(txt, ": ");
        if (sep)
            printf("%s[%s]%s %s%.*s%s: %s\n", COL(GRIS), h, COL(RESET),
                   COL(NEGRITA CIAN), (int)(sep - txt), txt, COL(RESET), sep + 2);
        else
            printf("[%s] %s\n", h, txt);
    } else if (strncmp(linea, "SYS ", 4) == 0) {
        printf("%s[%s] * %s%s\n", COL(AMARILLO), h, linea + 4, COL(RESET));
    } else if (strncmp(linea, "LST ", 4) == 0) {
        printf("%s[%s] %s%s\n", COL(VERDE), h, linea + 4, COL(RESET));
    } else if (strncmp(linea, "ERR ", 4) == 0) {
        printf("%s[%s] ! %s%s\n", COL(ROJO), h, linea + 4, COL(RESET));
    } else if (strncmp(linea, "OK ", 3) != 0) {
        printf("%s\n", linea);
    }
    mostrar_prompt();
    pthread_mutex_unlock(&mutex_pantalla);
}

/* Hilo receptor: muestra en tiempo real todo lo que llega del servidor */
static void *recibir(void *arg)
{
    (void)arg;
    char linea[MAX_LINEA + 64];
    int r;
    while ((r = leer_linea(fd_srv, &lector, linea, sizeof linea)) == 1)
        mostrar_del_servidor(linea);

```

```
    pthread_mutex_lock(&mutex_pantalla);
    if (interactivo) printf("\r\033[K");
    if (saliendo) {
        printf("Desconectado del servidor.\n");
        fflush(stdout);
        close(fd_srv);                                          /* close() */
        exit(0);
    }
    printf("%s! Se perdió la conexión con el servidor (%s).%s\n", COL(ROJO),
           r == 0 ? "el servidor cerró la conexión" : strerror(errno), COL(RESET));
    fflush(stdout);
    close(fd_srv);
    exit(1);
}

/* Ctrl+C: avisa al servidor con /quitar en lugar de cortar de golpe.
 * send() es seguro de usar dentro de un manejador de señales. */
static void al_interrumpir(int sig)
{
    (void)sig;
    saliendo = 1;
    if (fd_srv >= 0) send(fd_srv, "/quitar\n", 8, MSG_NOSIGNAL);
}

/* Resuelve el host y se conecta con un tiempo máximo de espera.
 * Devuelve el descriptor del socket conectado o -1. */
static int conectar(const char *host, const char *puerto)
{
    struct addrinfo pistas, *res, *p;
    memset(&pistas, 0, sizeof pistas);
    pistas.ai_family   = AF_INET;          /* IPv4 */
    pistas.ai_socktype = SOCK_STREAM;      /* TCP  */

    int e = getaddrinfo(host, puerto, &pistas, &res);
    if (e != 0) {
        fprintf(stderr, "No se pudo resolver '%s': %s\n", host, gai_strerror(e));
        return -1;
    }

    int fd = -1, err = 0;
    for (p = res; p; p = p->ai_next) {
        /* (1) socket(): crea el socket TCP del cliente */
        fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
        if (fd < 0) {
            err = errno;
            continue;
        }

        /* (2) connect(): inicia el three-way handshake (SYN, SYN-ACK, ACK).
         * Se hace no bloqueante para poder limitar la espera con select(). */
        int flags = fcntl(fd, F_GETFL, 0);
        fcntl(fd, F_SETFL, flags | O_NONBLOCK);
        int r = connect(fd, p->ai_addr, p->ai_addrlen);
        if (r < 0 && errno == EINPROGRESS) {
            fd_set escritura;
            FD_ZERO(&escritura);
            FD_SET(fd, &escritura);
            struct timeval tv = { .tv_sec = TIMEOUT_CONEXION, .tv_usec = 0 };
            r = select(fd + 1, NULL, &escritura, NULL, &tv);
            if (r == 0) {
                err = ETIMEDOUT;
                r = -1;
            } else if (r > 0) {
                socklen_t l = sizeof err;
                getsockopt(fd, SOL_SOCKET, SO_ERROR, &err, &l);
                r = err ? -1 : 0;
            } else {
                err = errno;
            }
        } else if (r < 0) {
            err = errno;
        }

        if (r == 0) {
            fcntl(fd, F_SETFL, flags);      /* volver a modo bloqueante */
            break;
        }
        close(fd);
        fd = -1;
    }
    freeaddrinfo(res);

    if (fd < 0) {
        if (err == ECONNREFUSED)
            fprintf(stderr, "Conexión rechazada: no hay ningún servidor escuchando en %s:%s\n", host, puerto);
        else if (err == ETIMEDOUT)
            fprintf(stderr, "Tiempo de espera agotado (%d s): el host %s no responde. "
                            "¿La IP es correcta? ¿Hay un firewall?\n", TIMEOUT_CONEXION, host);
```

```
        else if (err == EHOSTUNREACH || err == ENETUNREACH)
            fprintf(stderr, "Host o red inalcanzable: %s\n", host);
        else
            fprintf(stderr, "No se pudo conectar: %s\n", strerror(err));
    }
    return fd;
}

int main(int argc, char *argv[])
{
    char host[256], puerto[16];
    interactivo = isatty(STDIN_FILENO) && isatty(STDOUT_FILENO);
    signal(SIGPIPE, SIG_IGN);

    printf("=== Cliente de chat TCP - Redes de Datos (UNNE) ===\n");
    if (argc >= 2) snprintf(host, sizeof host, "%s", argv[1]);
    else pedir("IP o nombre del servidor [" HOST_DEFECTO "]: ", host, sizeof host, HOST_DEFECTO);
    if (argc >= 3) snprintf(puerto, sizeof puerto, "%s", argv[2]);
    else pedir("Puerto del servidor [" PUERTO_DEFECTO "]: ", puerto, sizeof puerto, PUERTO_DEFECTO);

    char *fin;
    long p = strtol(puerto, &fin, 10);
    if (*fin != '\0' || p < 1 || p > 65535) {
        fprintf(stderr, "Puerto inválido: '%s' (debe ser un número entre 1 y 65535)\n", puerto);
        return 1;
    }

    printf("Conectando a %s:%s ...\n", host, puerto);
    fd_srv = conectar(host, puerto);
    if (fd_srv < 0) return 1;

    /* Mostrar la tupla de la conexión (útil para comparar con Wireshark) */
    struct sockaddr_in loc, rem;
    socklen_t l1 = sizeof loc, l2 = sizeof rem;
    getsockname(fd_srv, (struct sockaddr *)&loc, &l1);
    getpeername(fd_srv, (struct sockaddr *)&rem, &l2);
    char ip_loc[INET_ADDRSTRLEN], ip_rem[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &loc.sin_addr, ip_loc, sizeof ip_loc);
    inet_ntop(AF_INET, &rem.sin_addr, ip_rem, sizeof ip_rem);
    printf("%sConexión exitosa%s: %s:%d (cliente) -> %s:%d (servidor)\n",
           COL(VERDE NEGRITA), COL(RESET), ip_loc, ntohs(loc.sin_port), ip_rem, ntohs(rem.sin_port));

    /* Bienvenida y registro del nombre de usuario */
    char linea[MAX_LINEA + 64];
    if (leer_linea(fd_srv, &lector, linea, sizeof linea) <= 0) {
        fprintf(stderr, "El servidor cerró la conexión.\n");
        return 1;
    }
    if (strncmp(linea, "SYS ", 4) == 0 || strncmp(linea, "ERR ", 4) == 0)
        printf("%s\n", linea + 4);

    for (;;) {
        char nombre[64];
        pedir("Nombre de usuario: ", nombre, sizeof nombre, NULL);
        if (nombre[0] == '\0') continue;
        if (enviar_linea(fd_srv, nombre) < 0) {
            fprintf(stderr, "Error al enviar el nombre: %s\n", strerror(errno));
            return 1;
        }
        int r = leer_linea(fd_srv, &lector, linea, sizeof linea);
        if (r <= 0) {
            fprintf(stderr, "El servidor cerró la conexión.\n");
            return 1;
        }
        if (strncmp(linea, "OK ", 3) == 0) {
            snprintf(mi_nombre, sizeof mi_nombre, "%.31s", linea + 3);   /* el servidor limita a 20 */
            break;
        }
        printf("%s! %s%s\n", COL(ROJO), strncmp(linea, "ERR ", 4) == 0 ? linea + 4 : linea, COL(RESET));
    }

    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = al_interrumpir;        /* sin SA_RESTART: fgets() se interrumpe */
    sigemptyset(&sa.sa_mask);
    sigaction(SIGINT, &sa, NULL);

    pthread_t hilo;
    if (pthread_create(&hilo, NULL, recibir, NULL) != 0) {
        fprintf(stderr, "No se pudo crear el hilo receptor\n");
        return 1;
    }

    /* Hilo principal: leer teclado y enviar */
    char entrada[2048];
    pthread_mutex_lock(&mutex_pantalla);
    mostrar_prompt();
    pthread_mutex_unlock(&mutex_pantalla);
```

```
368
369     while (!saliendo && fgets(entrada, sizeof entrada, stdin)) {
370         size_t n = strlen(entrada);
371         int demasiado_larga = 0;
372         if (n > 0 && entrada[n - 1] != '\n' && !feof(stdin)) {
373             int ch;
374             while ((ch = getchar()) != '\n' && ch != EOF) {}
375             demasiado_larga = 1;
376         }
377         entrada[strcspn(entrada, "\r\n")] = '\0';
378         recortar(entrada);
379
380         if (entrada[0] == '\0' || demasiado_larga || strlen(entrada) > MAX_MENSAJE) {
381             pthread_mutex_lock(&mutex_pantalla);
382             if (entrada[0] != '\0')
383                 printf("%s! Mensaje demasiado largo (máximo %d caracteres)%s\n",
384                        COL(ROJO), MAX_MENSAJE, COL(RESET));
385             mostrar_prompt();
386             pthread_mutex_unlock(&mutex_pantalla);
387             continue;
388         }
389
390         if (strcasecmp(entrada, "/quitar") == 0) saliendo = 1;
391         if (enviar_linea(fd_srv, entrada) < 0) {
392             fprintf(stderr, "Error al enviar: se perdió la conexión con el servidor\n");
393             return 1;
394         }
395         if (saliendo) break;
396
397         pthread_mutex_lock(&mutex_pantalla);
398         mostrar_prompt();
399         pthread_mutex_unlock(&mutex_pantalla);
400     }
401
402     if (!saliendo) {                      /* Ctrl+D (fin de entrada) */
403         saliendo = 1;
404         enviar_linea(fd_srv, "/quitar");
405     }
406     pthread_join(hilo, NULL);             /* el receptor termina cuando el servidor cierra */
407     return 0;
408 }
```

Anexo B — Código fuente del Ejercicio 2

B.1. servidor_inventario.c

```c
/*
 * servidor_inventario.c  -  TP 3.2 Redes de Datos (UNNE FaCENA) - Ejercicio 2, escenario 2
 *
 * Servidor de consultas de características HW/SW para inventario.
 * Los equipos a inventariar ejecutan "agente_inventario" y se conectan a este
 * servidor. Desde el menú del servidor se elige un equipo y se le envían
 * consultas (CPU, RAM, disco, filesystem, loadavg, particiones, SO, red,
 * procesos); el agente las responde leyendo /proc y comandos del sistema.
 *
 * Compilar:  gcc -Wall -Wextra -pthread -o servidor_inventario servidor_inventario.c
 * Ejecutar:  ./servidor_inventario [puerto]         (por defecto 6000)
 *
 * Protocolo de aplicación (sobre TCP):
 *   Agente -> Servidor : HOLA <hostname>\n           (al conectarse)
 *   Servidor -> Agente : OK <id>\n
 *   Servidor -> Agente : CONSULTA <tipo>\n
 *   Agente -> Servidor : RESP <n>\n<n bytes de texto>   o   ERR <n>\n<n bytes>
 *   Servidor -> Agente : FIN\n                       (al cerrar el servidor)
 * La respuesta lleva su longitud en bytes porque puede tener muchas líneas:
 * así el servidor sabe exactamente cuántos bytes leer del flujo TCP.
 */
#define _GNU_SOURCE
#include <arpa/inet.h>
#include <ctype.h>
#include <errno.h>
#include <ifaddrs.h>
#include <netinet/in.h>
#include <pthread.h>
#include <signal.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <time.h>
#include <unistd.h>

#define PUERTO_DEFECTO     6000
#define MAX_EQUIPOS        32
#define MAX_LINEA          512
#define MAX_RESPUESTA      (8 * 1024 * 1024)
#define TIMEOUT_SALUDO     5     /* segundos para recibir HOLA */
#define TIMEOUT_RESPUESTA  15    /* segundos para recibir cada respuesta */

typedef struct {
    char   datos[MAX_LINEA * 2];
    size_t len;
} lector_t;

typedef struct {
    int      fd, id, puerto;
    char     host[64];
    char     ip[INET_ADDRSTRLEN];
    time_t   desde;
    lector_t lr;
} equipo_t;

static const struct {
    const char *clave;
    const char *descripcion;
} CONSULTAS[] = {
    { "cpu",         "Características del procesador" },
    { "ram",         "Memoria RAM y swap" },
    { "disco",       "Estado de los discos (uso y espacio libre)" },
    { "fs",          "Sistemas de archivos montados" },
    { "loadavg",     "Promedio de carga (loadavg) y uptime" },
    { "particiones", "Particiones" },
    { "so",          "Versión del sistema operativo y hostname" },
    { "red",         "Interfaces de red" },
    { "procesos",    "Procesos en ejecución" },
};
#define N_CONSULTAS ((int)(sizeof CONSULTAS / sizeof CONSULTAS[0]))

static equipo_t *equipos[MAX_EQUIPOS];
static int proximo_id = 1;
static int fd_escucha = -1;
static pthread_mutex_t mutex_equipos  = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t mutex_pantalla = PTHREAD_MUTEX_INITIALIZER;

/* ------------------------------------------------------------------ */
/* Utilidades de sockets                                              */
```

```c
/* ------------------------------------------------------------------ */

static int enviar_todo(int fd, const char *buf, size_t len)
{
    while (len > 0) {
        ssize_t n = send(fd, buf, len, MSG_NOSIGNAL);            /* send() */
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        buf += n;
        len -= (size_t)n;
    }
    return 0;
}

static int enviar_fmt(int fd, const char *fmt, ...)
{
    char buf[MAX_LINEA + 2];
    va_list ap;
    va_start(ap, fmt);
    int n = vsnprintf(buf, sizeof buf - 1, fmt, ap);
    va_end(ap);
    if (n < 0) return -1;
    if ((size_t)n > sizeof buf - 2) n = (int)(sizeof buf - 2);
    buf[n++] = '\n';
    return enviar_todo(fd, buf, (size_t)n);
}

/* 1 = línea, 0 = el agente cerró, -1 = error o tiempo agotado */
static int leer_linea(int fd, lector_t *lr, char *linea, size_t tam)
{
    for (;;) {
        char  *nl = memchr(lr->datos, '\n', lr->len);
        size_t n;
        if (nl)
            n = (size_t)(nl - lr->datos);
        else if (lr->len == sizeof lr->datos)
            n = lr->len;
        else {
            ssize_t r = recv(fd, lr->datos + lr->len,                 /* recv() */
                             sizeof lr->datos - lr->len, 0);
            if (r == 0) return 0;
            if (r < 0) {
                if (errno == EINTR) continue;
                return -1;
            }
            lr->len += (size_t)r;
            continue;
        }
        size_t copiar = n < tam - 1 ? n : tam - 1;
        memcpy(linea, lr->datos, copiar);
        linea[copiar] = '\0';
        size_t consumir = nl ? n + 1 : n;
        memmove(lr->datos, lr->datos + consumir, lr->len - consumir);
        lr->len -= consumir;
        if (copiar > 0 && linea[copiar - 1] == '\r') linea[copiar - 1] = '\0';
        return 1;
    }
}

/* Lee exactamente n bytes (primero lo que quedó en el buffer, luego recv) */
static int leer_bloque(int fd, lector_t *lr, char *dst, size_t n)
{
    size_t del_buffer = lr->len < n ? lr->len : n;
    memcpy(dst, lr->datos, del_buffer);
    memmove(lr->datos, lr->datos + del_buffer, lr->len - del_buffer);
    lr->len -= del_buffer;

    size_t hecho = del_buffer;
    while (hecho < n) {
        ssize_t r = recv(fd, dst + hecho, n - hecho, 0);             /* recv() */
        if (r == 0) return 0;
        if (r < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        hecho += (size_t)r;
    }
    return 1;
}

static void avisar(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    pthread_mutex_lock(&mutex_pantalla);
    printf("\n");
```

```
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    pthread_mutex_unlock(&mutex_pantalla);
    va_end(ap);
}

/* ------------------------------------------------------------------ */
/* Gestión de equipos conectados                                       */
/* ------------------------------------------------------------------ */

static void desconectar_equipo(equipo_t *e, const char *motivo)
{
    pthread_mutex_lock(&mutex_equipos);
    for (int i = 0; i < MAX_EQUIPOS; i++)
        if (equipos[i] == e) equipos[i] = NULL;
    pthread_mutex_unlock(&mutex_equipos);
    avisar("[-] Equipo #%d %s (%s:%d) desconectado: %s", e->id, e->host, e->ip, e->puerto, motivo);
    close(e->fd);                                               /* close() */
    free(e);
}

/* Detecta agentes que se cayeron mientras estaban inactivos:
 * recv con MSG_PEEK no consume datos; si devuelve 0 el otro extremo cerró. */
static void verificar_conexiones(void)
{
    equipo_t *caidos[MAX_EQUIPOS];
    int n = 0;
    pthread_mutex_lock(&mutex_equipos);
    for (int i = 0; i < MAX_EQUIPOS; i++) {
        if (!equipos[i]) continue;
        char c;
        ssize_t r = recv(equipos[i]->fd, &c, 1, MSG_PEEK | MSG_DONTWAIT);
        if (r == 0 || (r < 0 && errno != EAGAIN && errno != EWOULDBLOCK))
            caidos[n++] = equipos[i];
    }
    pthread_mutex_unlock(&mutex_equipos);
    for (int i = 0; i < n; i++)
        desconectar_equipo(caidos[i], "la conexión se cerró");
}

static equipo_t *buscar_equipo(int id)
{
    equipo_t *e = NULL;
    pthread_mutex_lock(&mutex_equipos);
    for (int i = 0; i < MAX_EQUIPOS; i++)
        if (equipos[i] && equipos[i]->id == id) e = equipos[i];
    pthread_mutex_unlock(&mutex_equipos);
    return e;
}

/* Envía una consulta y recibe la respuesta completa.
 * 0 = ok, 1 = el agente respondió con error, -1 = conexión perdida */
static int consultar(equipo_t *e, const char *tipo, char **respuesta)
{
    char linea[MAX_LINEA];
    *respuesta = NULL;

    if (enviar_fmt(e->fd, "CONSULTA %s", tipo) < 0) return -1;

    int r = leer_linea(e->fd, &e->lr, linea, sizeof linea);
    if (r <= 0) return -1;

    int es_error = 0;
    long n;
    if (strncmp(linea, "RESP ", 5) == 0) {
        n = strtol(linea + 5, NULL, 10);
    } else if (strncmp(linea, "ERR ", 4) == 0) {
        es_error = 1;
        n = strtol(linea + 4, NULL, 10);
    } else {
        return -1;                                      /* respuesta inválida */
    }
    if (n < 0 || n > MAX_RESPUESTA) return -1;

    *respuesta = malloc((size_t)n + 1);
    if (!*respuesta) return -1;
    if (leer_bloque(e->fd, &e->lr, *respuesta, (size_t)n) <= 0) {
        free(*respuesta);
        *respuesta = NULL;
        return -1;
    }
    (*respuesta)[n] = '\0';
    return es_error;
}

/* Hilo que acepta agentes en segundo plano mientras el menú sigue activo */
static void *aceptar_equipos(void *arg)
```

```c
{
    (void)arg;
    for (;;) {
        struct sockaddr_in cli;
        socklen_t lc = sizeof cli;
        /* (4) accept(): espera y acepta la conexión de un agente */
        int fd = accept(fd_escucha, (struct sockaddr *)&cli, &lc);
        if (fd < 0) {
            if (errno == EINTR || errno == ECONNABORTED) continue;
            if (errno == EBADF || errno == EINVAL) break;  /* socket cerrado */
            avisar("[!] accept falló: %s", strerror(errno));
            sleep(1);
            continue;
        }

        equipo_t *e = calloc(1, sizeof *e);
        if (!e) {
            close(fd);
            continue;
        }
        e->fd = fd;
        e->desde = time(NULL);
        inet_ntop(AF_INET, &cli.sin_addr, e->ip, sizeof e->ip);
        e->puerto = ntohs(cli.sin_port);

        /* Tiempo máximo de espera para el saludo y luego para cada respuesta */
        struct timeval tv = { .tv_sec = TIMEOUT_SALUDO, .tv_usec = 0 };
        setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
        setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &tv, sizeof tv);

        char linea[MAX_LINEA];
        int r = leer_linea(fd, &e->lr, linea, sizeof linea);
        if (r <= 0 || strncmp(linea, "HOLA ", 5) != 0) {
            enviar_fmt(fd, "ERR Protocolo inválido: se esperaba 'HOLA <hostname>'");
            avisar("[!] Conexión de %s:%d rechazada: no es un agente de inventario", e->ip, e->puerto);
            close(fd);
            free(e);
            continue;
        }
        /* Copiar el hostname aceptando solo caracteres seguros */
        size_t j = 0;
        for (const char *s = linea + 5; *s && j < sizeof e->host - 1; s++)
            if (isalnum((unsigned char)*s) || *s == '-' || *s == '_' || *s == '.')
                e->host[j++] = *s;
        e->host[j] = '\0';
        if (j == 0) snprintf(e->host, sizeof e->host, "desconocido");

        int lugar = -1;
        pthread_mutex_lock(&mutex_equipos);
        for (int i = 0; i < MAX_EQUIPOS && lugar < 0; i++)
            if (!equipos[i]) lugar = i;
        if (lugar >= 0) {
            e->id = proximo_id++;
            equipos[lugar] = e;
        }
        pthread_mutex_unlock(&mutex_equipos);

        if (lugar < 0) {
            enviar_fmt(fd, "ERR Servidor lleno (máximo %d equipos)", MAX_EQUIPOS);
            avisar("[!] %s rechazado: servidor lleno", e->host);
            close(fd);
            free(e);
            continue;
        }

        tv.tv_sec = TIMEOUT_RESPUESTA;
        setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
        enviar_fmt(fd, "OK %d", e->id);
        avisar("[+] Equipo #%d conectado: %s (%s:%d)", e->id, e->host, e->ip, e->puerto);
    }
    return NULL;
}

/* ------------------------------------------------------------------ */
/* Menú                                                               */
/* ------------------------------------------------------------------ */

static void apagar(void)
{
    pthread_mutex_lock(&mutex_equipos);
    for (int i = 0; i < MAX_EQUIPOS; i++) {
        if (!equipos[i]) continue;
        enviar_fmt(equipos[i]->fd, "FIN");
        close(equipos[i]->fd);
    }
    pthread_mutex_unlock(&mutex_equipos);
    shutdown(fd_escucha, SHUT_RDWR);
    close(fd_escucha);
```

```
    printf("Servidor detenido.\n");
    exit(0);
}

/* Lee un entero del teclado. Devuelve -1 si la entrada no es un número. */
static int leer_entero(const char *mensaje)
{
    char buf[64];
    pthread_mutex_lock(&mutex_pantalla);
    printf("%s", mensaje);
    fflush(stdout);
    pthread_mutex_unlock(&mutex_pantalla);
    if (!fgets(buf, sizeof buf, stdin)) apagar();     /* Ctrl+D */
    char *fin;
    long v = strtol(buf, &fin, 10);
    while (*fin == ' ' || *fin == '\n' || *fin == '\r') fin++;
    if (fin == buf || *fin != '\0') return -1;
    return (int)v;
}

static int listar_equipos(void)
{
    verificar_conexiones();
    int n = 0;
    time_t ahora = time(NULL);
    pthread_mutex_lock(&mutex_equipos);
    pthread_mutex_lock(&mutex_pantalla);
    printf("\n  %-4s %-22s %-22s %s\n", "ID", "Hostname", "IP:puerto (agente)", "Conectado hace");
    for (int i = 0; i < MAX_EQUIPOS; i++) {
        equipo_t *e = equipos[i];
        if (!e) continue;
        char dir[40];
        snprintf(dir, sizeof dir, "%s:%d", e->ip, e->puerto);
        long s = (long)(ahora - e->desde);
        printf("  #%-3d %-22s %-22s %ldm %02lds\n", e->id, e->host, dir, s / 60, s % 60);
        n++;
    }
    if (n == 0) printf("  (no hay equipos conectados: ejecute agente_inventario en los clientes)\n");
    pthread_mutex_unlock(&mutex_pantalla);
    pthread_mutex_unlock(&mutex_equipos);
    return n;
}

static equipo_t *elegir_equipo(void)
{
    if (listar_equipos() == 0) return NULL;
    int id = leer_entero("ID del equipo: ");
    equipo_t *e = buscar_equipo(id);
    if (!e) printf("No existe un equipo con ese ID.\n");
    return e;
}

static int elegir_consulta(void)
{
    printf("\n");
    for (int i = 0; i < N_CONSULTAS; i++)
        printf("  %d) %-12s %s\n", i + 1, CONSULTAS[i].clave, CONSULTAS[i].descripcion);
    int op = leer_entero("Consulta: ");
    if (op < 1 || op > N_CONSULTAS) {
        printf("Opción inválida.\n");
        return -1;
    }
    return op - 1;
}

static void imprimir_resultado(FILE *f, const equipo_t *e, int c, const char *texto, int es_error)
{
    fprintf(f, "\n========== %s @ %s (%s) ==========\n", CONSULTAS[c].descripcion, e->host, e->ip);
    fprintf(f, "%s%s", es_error ? "ERROR: " : "", texto);
    if (texto[0] && texto[strlen(texto) - 1] != '\n') fprintf(f, "\n");
}

/* Ejecuta una consulta y muestra el resultado. Devuelve -1 si se perdió el equipo. */
static int ejecutar_consulta(equipo_t *e, int c, FILE *copia)
{
    char *resp;
    int r = consultar(e, CONSULTAS[c].clave, &resp);
    if (r < 0) {
        desconectar_equipo(e, errno == EAGAIN || errno == EWOULDBLOCK
                                  ? "no respondió a tiempo" : "se perdió la conexión");
        return -1;
    }
    pthread_mutex_lock(&mutex_pantalla);
    imprimir_resultado(stdout, e, c, resp, r);
    pthread_mutex_unlock(&mutex_pantalla);
    if (copia) imprimir_resultado(copia, e, c, resp, r);
    free(resp);
    return 0;
```

```
}

static void inventario_completo(equipo_t *e)
{
    char nombre[128], fecha[32];
    time_t t = time(NULL);
    struct tm tm;
    localtime_r(&t, &tm);
    strftime(fecha, sizeof fecha, "%Y%m%d_%H%M%S", &tm);
    snprintf(nombre, sizeof nombre, "inventario_%s_%s.txt", e->host, fecha);

    FILE *f = fopen(nombre, "w");
    if (f) {
        strftime(fecha, sizeof fecha, "%d/%m/%Y %H:%M:%S", &tm);
        fprintf(f, "INVENTARIO HW/SW - equipo %s (%s) - %s\n", e->host, e->ip, fecha);
    }
    for (int c = 0; c < N_CONSULTAS; c++)
        if (ejecutar_consulta(e, c, f) < 0) break;
    if (f) {
        fclose(f);
        printf("\nInventario guardado en: %s\n", nombre);
    }
}

static void consultar_todos(void)
{
    if (listar_equipos() == 0) return;
    int c = elegir_consulta();
    if (c < 0) return;
    int ids[MAX_EQUIPOS], n = 0;
    pthread_mutex_lock(&mutex_equipos);
    for (int i = 0; i < MAX_EQUIPOS; i++)
        if (equipos[i]) ids[n++] = equipos[i]->id;
    pthread_mutex_unlock(&mutex_equipos);
    for (int i = 0; i < n; i++) {
        equipo_t *e = buscar_equipo(ids[i]);
        if (e) ejecutar_consulta(e, c, NULL);
    }
}

static void mostrar_direcciones(int puerto)
{
    struct ifaddrs *ifs, *i;
    printf("Servidor de inventario escuchando en el puerto TCP %d. Direcciones de este equipo:\n", puerto);
    if (getifaddrs(&ifs) == 0) {
        for (i = ifs; i; i = i->ifa_next) {
            if (!i->ifa_addr || i->ifa_addr->sa_family != AF_INET) continue;
            char ip[INET_ADDRSTRLEN];
            inet_ntop(AF_INET, &((struct sockaddr_in *)i->ifa_addr)->sin_addr, ip, sizeof ip);
            printf("   %-10s %s:%d\n", i->ifa_name, ip, puerto);
        }
        freeifaddrs(ifs);
    }
}

int main(int argc, char *argv[])
{
    int puerto = PUERTO_DEFECTO;
    if (argc > 1) {
        char *fin;
        long p = strtol(argv[1], &fin, 10);
        if (*fin != '\0' || p < 1 || p > 65535) {
            fprintf(stderr, "Puerto inválido: '%s'. Uso: %s [puerto 1-65535]\n", argv[1], argv[0]);
            return 1;
        }
        puerto = (int)p;
    }
    signal(SIGPIPE, SIG_IGN);

    /* (1) socket() TCP/IPv4 */
    fd_escucha = socket(AF_INET, SOCK_STREAM, 0);
    if (fd_escucha < 0) {
        perror("socket");
        return 1;
    }
    int si = 1;
    setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof si);

    /* (2) bind() a 0.0.0.0:puerto */
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof dir);
    dir.sin_family      = AF_INET;
    dir.sin_addr.s_addr = htonl(INADDR_ANY);
    dir.sin_port        = htons((uint16_t)puerto);
    if (bind(fd_escucha, (struct sockaddr *)&dir, sizeof dir) < 0) {
        if (errno == EADDRINUSE)
            fprintf(stderr, "Error: el puerto %d ya está en uso. Pruebe con: %s %d\n",
                    puerto, argv[0], puerto + 1);
```

```
        else
            perror("bind");
        return 1;
    }

    /* (3) listen() */
    if (listen(fd_escucha, 10) < 0) {
        perror("listen");
        return 1;
    }
    mostrar_direcciones(puerto);

    pthread_t hilo;
    if (pthread_create(&hilo, NULL, aceptar_equipos, NULL) != 0) {
        fprintf(stderr, "No se pudo crear el hilo aceptador\n");
        return 1;
    }
    pthread_detach(hilo);

    for (;;) {
        pthread_mutex_lock(&mutex_pantalla);
        printf("\n===== SERVIDOR DE INVENTARIO HW/SW =====\n"
               "  1) Listar equipos conectados\n"
               "  2) Consultar un equipo\n"
               "  3) Inventario completo de un equipo (y guardar en archivo)\n"
               "  4) Hacer una consulta a todos los equipos\n"
               "  0) Salir\n");
        pthread_mutex_unlock(&mutex_pantalla);
        int op = leer_entero("Opción: ");
        equipo_t *e;
        int c;
        switch (op) {
        case 1:
            listar_equipos();
            break;
        case 2:
            if ((e = elegir_equipo()) && (c = elegir_consulta()) >= 0)
                ejecutar_consulta(e, c, NULL);
            break;
        case 3:
            if ((e = elegir_equipo())) inventario_completo(e);
            break;
        case 4:
            consultar_todos();
            break;
        case 0:
            apagar();
            break;
        default:
            printf("Opción inválida.\n");
        }
    }
}
```

B.2. agente_inventario.c

```
/*
 * agente_inventario.c  -  TP 3.2 Redes de Datos (UNNE FaCENA) - Ejercicio 2, escenario 2
 *
 * Cliente (agente) que se instala en el equipo a inventariar. Se conecta al
 * servidor de inventario, queda esperando consultas y responde cada una con
 * datos de hardware y software leídos de /proc, /sys, /etc y comandos del
 * sistema (df, lsblk, free, ps, hostnamectl).
 *
 * Seguridad: el agente SOLO responde a la lista fija de consultas definida
 * abajo. Nunca ejecuta comandos arbitrarios enviados por la red (eso lo
 * convertiría en una puerta trasera / shell remota).
 *
 * Compilar:  gcc -Wall -Wextra -o agente_inventario agente_inventario.c
 * Ejecutar:  ./agente_inventario [host] [puerto]   (si faltan, se piden por teclado)
 */
#define _GNU_SOURCE
#include <arpa/inet.h>
#include <ctype.h>
#include <dirent.h>
#include <errno.h>
#include <fcntl.h>
#include <ifaddrs.h>
#include <netdb.h>
#include <net/if.h>
#include <signal.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <sys/socket.h>
```

```c
#include <sys/statvfs.h>
#include <sys/utsname.h>
#include <time.h>
#include <unistd.h>

#define HOST_DEFECTO      "127.0.0.1"
#define PUERTO_DEFECTO    "6000"
#define MAX_LINEA         512
#define TIMEOUT_CONEXION  5

typedef struct {
    char   datos[MAX_LINEA * 2];
    size_t len;
} lector_t;

/* Buffer de texto dinámico donde se arma cada respuesta */
typedef struct {
    char  *d;
    size_t len, cap;
} buf_t;

/* ------------------------------------------------------------------ */
/* Buffer de texto                                                     */
/* ------------------------------------------------------------------ */

static void b_reservar(buf_t *b, size_t extra)
{
    if (b->len + extra + 1 <= b->cap) return;
    size_t nueva = b->cap ? b->cap : 4096;
    while (nueva < b->len + extra + 1) nueva *= 2;
    char *p = realloc(b->d, nueva);
    if (!p) {
        perror("realloc");
        exit(1);
    }
    b->d = p;
    b->cap = nueva;
}

static void b_printf(buf_t *b, const char *fmt, ...)
{
    va_list ap, ap2;
    va_start(ap, fmt);
    va_copy(ap2, ap);
    int n = vsnprintf(NULL, 0, fmt, ap);
    va_end(ap);
    if (n > 0) {
        b_reservar(b, (size_t)n);
        vsnprintf(b->d + b->len, (size_t)n + 1, fmt, ap2);
        b->len += (size_t)n;
    }
    va_end(ap2);
}

static void b_agregar_flujo(buf_t *b, FILE *f)
{
    char tmp[4096];
    size_t n;
    while ((n = fread(tmp, 1, sizeof tmp, f)) > 0) {
        b_reservar(b, n);
        memcpy(b->d + b->len, tmp, n);
        b->len += n;
        b->d[b->len] = '\0';
    }
}

/* Agrega el contenido completo de un archivo (ej.: /proc/partitions) */
static void b_archivo(buf_t *b, const char *ruta)
{
    FILE *f = fopen(ruta, "r");
    if (!f) {
        b_printf(b, "(no se pudo leer %s: %s)\n", ruta, strerror(errno));
        return;
    }
    b_agregar_flujo(b, f);
    fclose(f);
}

/* Agrega la salida de un comando FIJO del sistema (nunca uno recibido por red) */
static void b_comando(buf_t *b, const char *titulo, const char *cmd)
{
    b_printf(b, "\n$ %s\n", titulo);
    FILE *p = popen(cmd, "r");
    if (!p) {
        b_printf(b, "(no se pudo ejecutar)\n");
        return;
    }
    size_t antes = b->len;
```

```
    b_agregar_flujo(b, p);
    pclose(p);
    if (b->len == antes) b_printf(b, "(comando no disponible en este sistema)\n");
}

/* Escribe "  etiqueta" rellenada a 'ancho' caracteres visibles (printf rellena por
 * bytes y las letras con tilde ocupan 2 bytes en UTF-8, lo que desalinea) */
static void b_etiqueta(buf_t *b, const char *et, int ancho)
{
    int visibles = 0;
    for (const char *p = et; *p; p++)
        if (((unsigned char)*p & 0xC0) != 0x80) visibles++;
    b_printf(b, "  %s%*s ", et, ancho > visibles ? ancho - visibles : 0, "");
}

/* Busca "clave : valor" en un archivo tipo /proc/cpuinfo o /proc/meminfo */
static int buscar_campo(const char *ruta, const char *clave, char *valor, size_t tam)
{
    FILE *f = fopen(ruta, "r");
    if (!f) return 0;
    char linea[1024];
    size_t lc = strlen(clave);
    int hallado = 0;
    while (fgets(linea, sizeof linea, f)) {
        if (strncmp(linea, clave, lc) != 0) continue;
        char *p = linea + lc;
        if (*p != ' ' && *p != '\t' && *p != ':') continue;
        p = strchr(p, ':');
        if (!p) continue;
        p++;
        while (*p == ' ' || *p == '\t') p++;
        p[strcspn(p, "\n")] = '\0';
        snprintf(valor, tam, "%s", p);
        hallado = 1;
        break;
    }
    fclose(f);
    return hallado;
}

static long meminfo_kb(const char *clave)
{
    char v[64];
    if (!buscar_campo("/proc/meminfo", clave, v, sizeof v)) return -1;
    return strtol(v, NULL, 10);
}

/* ------------------------------------------------------------------ */
/* Consultas                                                          */
/* ------------------------------------------------------------------ */

static void q_cpu(buf_t *b)
{
    const char *campos[][2] = {
        { "model name", "Modelo" },          { "vendor_id", "Fabricante" },
        { "cpu MHz", "Frecuencia (MHz)" },   { "cache size", "Caché" },
        { "cpu cores", "Núcleos físicos" },  { "siblings", "Hilos por socket" },
        { "Hardware", "Hardware (ARM)" },    { "Model", "Modelo (ARM)" },
    };
    char v[512];
    b_printf(b, "Fuente: /proc/cpuinfo\n");
    for (size_t i = 0; i < sizeof campos / sizeof campos[0]; i++)
        if (buscar_campo("/proc/cpuinfo", campos[i][0], v, sizeof v)) {
            b_etiqueta(b, campos[i][1], 20);
            b_printf(b, "%s\n", v);
        }

    int logicos = 0;
    FILE *f = fopen("/proc/cpuinfo", "r");
    if (f) {
        char l[1024];
        while (fgets(l, sizeof l, f))
            if (strncmp(l, "processor", 9) == 0) logicos++;
        fclose(f);
    }
    b_etiqueta(b, "CPUs lógicas", 20); b_printf(b, "%d\n", logicos);
    b_etiqueta(b, "CPUs en línea", 20); b_printf(b, "%ld\n", sysconf(_SC_NPROCESSORS_ONLN));
    struct utsname u;
    if (uname(&u) == 0) { b_etiqueta(b, "Arquitectura", 20); b_printf(b, "%s\n", u.machine); }
}

static void q_ram(buf_t *b)
{
    long total = meminfo_kb("MemTotal"), libre = meminfo_kb("MemFree");
    long disp = meminfo_kb("MemAvailable"), buffers = meminfo_kb("Buffers");
    long cache = meminfo_kb("Cached"), sw_t = meminfo_kb("SwapTotal"), sw_l = meminfo_kb("SwapFree");
    b_printf(b, "Fuente: /proc/meminfo\n");
    b_etiqueta(b, "Memoria total", 22); b_printf(b, "%8.1f MB\n", total / 1024.0);
```

```
    b_etiqueta(b, "Memoria libre", 22); b_printf(b, "%8.1f MB\n", libre / 1024.0);
    b_etiqueta(b, "Memoria disponible", 22); b_printf(b, "%8.1f MB\n", disp / 1024.0);
    b_etiqueta(b, "Buffers + caché", 22); b_printf(b, "%8.1f MB\n", (buffers + cache) / 1024.0);
    if (total > 0 && disp >= 0) {
        b_etiqueta(b, "Uso real", 22);
        b_printf(b, "%8.1f %%\n", 100.0 * (double)(total - disp) / (double)total);
    }
    b_etiqueta(b, "Swap total", 22); b_printf(b, "%8.1f MB (libre %.1f MB)\n", sw_t / 1024.0, sw_l / 1024.0);
    b_comando(b, "free -h", "free -h 2>/dev/null");
}

static void q_disco(buf_t *b)
{
    struct statvfs s;
    if (statvfs("/", &s) == 0) {
        double tam   = (double)s.f_blocks * s.f_frsize / 1e9;
        double libre = (double)s.f_bavail * s.f_frsize / 1e9;
        b_printf(b, "Partición raíz (/): %.1f GB totales, %.1f GB libres (%.1f %% usado)\n",
                 tam, libre, tam > 0 ? 100.0 * (tam - libre) / tam : 0.0);
    }
    b_comando(b, "lsblk -d -o NAME,SIZE,ROTA,TYPE,MODEL  (ROTA=1 disco rígido, 0 SSD)",
              "lsblk -d -o NAME,SIZE,ROTA,TYPE,MODEL 2>/dev/null");
    b_comando(b, "df -h (sin sistemas de archivos virtuales)",
              "df -h -x tmpfs -x devtmpfs -x squashfs -x overlay 2>/dev/null || df -h 2>/dev/null");
}

static void q_fs(buf_t *b)
{
    FILE *f = fopen("/proc/mounts", "r");
    b_printf(b, "Fuente: /proc/mounts (solo dispositivos /dev/*)\n");
    b_printf(b, "  %-24s %-22s %-8s %s\n", "Dispositivo", "Punto de montaje", "Tipo", "Modo");
    if (!f) {
        b_printf(b, "(no se pudo leer /proc/mounts)\n");
        return;
    }
    char dev[256], punto[256], tipo[64], opc[512];
    int n = 0;
    while (fscanf(f, "%255s %255s %63s %511s %*d %*d", dev, punto, tipo, opc) == 4) {
        if (strncmp(dev, "/dev/", 5) != 0) continue;
        b_printf(b, "  %-24s %-22s %-8s %s\n", dev, punto, tipo,
                            strncmp(opc, "ro", 2) == 0 && (opc[2] == ',' || opc[2] == '\0') ? "solo lectura" :
"lectura/escritura");
        n++;
    }
    fclose(f);
    if (n == 0) b_printf(b, "  (no hay dispositivos /dev montados: ¿contenedor?)\n");
    b_comando(b, "df -hT", "df -hT -x tmpfs -x devtmpfs -x squashfs 2>/dev/null");
}

static void q_loadavg(buf_t *b)
{
    double c1, c5, c15;
    char procs[32];
    long ultimo;
    FILE *f = fopen("/proc/loadavg", "r");
    if (!f || fscanf(f, "%lf %lf %lf %31s %ld", &c1, &c5, &c15, procs, &ultimo) != 5) {
        b_printf(b, "(no se pudo leer /proc/loadavg)\n");
        if (f) fclose(f);
        return;
    }
    fclose(f);
    long cpus = sysconf(_SC_NPROCESSORS_ONLN);
    b_printf(b, "Fuente: /proc/loadavg\n");
    b_printf(b, "  Carga promedio:  1 min = %.2f | 5 min = %.2f | 15 min = %.2f\n", c1, c5, c15);
    b_printf(b, "  Procesos (ejecutándose/total): %s   Último PID: %ld\n", procs, ultimo);
    b_printf(b, "  CPUs: %ld  ->  carga por CPU (1 min): %.2f  (%s)\n", cpus, c1 / (double)cpus,
             c1 / (double)cpus < 0.7 ? "holgado" : c1 / (double)cpus <= 1.0 ? "cerca del límite" : "sobrecargado");

    double up;
    f = fopen("/proc/uptime", "r");
    if (f && fscanf(f, "%lf", &up) == 1) {
        long s = (long)up;
        b_printf(b, "  Encendido hace: %ld días, %02ld:%02ld hs\n", s / 86400, (s % 86400) / 3600, (s % 3600) / 60);
    }
    if (f) fclose(f);
}

static void q_particiones(buf_t *b)
{
    b_printf(b, "Fuente: /proc/partitions (bloques de 1 KB)\n");
    b_archivo(b, "/proc/partitions");
    b_comando(b, "lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT",
              "lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT 2>/dev/null");
}

static void q_so(buf_t *b)
{
    char host[256] = "";
```

```
    gethostname(host, sizeof host);
    b_etiqueta(b, "Hostname", 18); b_printf(b, "%s\n", host);
    /* /etc/os-release tiene el formato CLAVE="valor" */
    FILE *f = fopen("/etc/os-release", "r");
    if (f) {
        char l[512];
        while (fgets(l, sizeof l, f))
            if (strncmp(l, "PRETTY_NAME=", 12) == 0) {
                char *v = l + 12;
                v[strcspn(v, "\n")] = '\0';
                if (*v == '"') {
                    v++;
                    v[strcspn(v, "\"")] = '\0';
                }
                b_etiqueta(b, "Distribución", 18); b_printf(b, "%s\n", v);
            }
        fclose(f);
    }
    struct utsname u;
    if (uname(&u) == 0) {
        b_etiqueta(b, "Kernel", 18); b_printf(b, "%s %s\n", u.sysname, u.release);
        b_etiqueta(b, "Compilación", 18); b_printf(b, "%s\n", u.version);
        b_etiqueta(b, "Arquitectura", 18); b_printf(b, "%s\n", u.machine);
    }
    b_comando(b, "hostnamectl", "hostnamectl 2>/dev/null");
}

static int prefijo_mascara(const struct sockaddr *m)
{
    int bits = 0;
    if (!m) return 0;
    const unsigned char *p;
    size_t n;
    if (m->sa_family == AF_INET) {
        p = (const unsigned char *)&((const struct sockaddr_in *)m)->sin_addr;
        n = 4;
    } else {
        p = (const unsigned char *)&((const struct sockaddr_in6 *)m)->sin6_addr;
        n = 16;
    }
    for (size_t i = 0; i < n; i++)
        for (int k = 7; k >= 0; k--)
            if (p[i] & (1 << k)) bits++;
    return bits;
}

static void leer_sys(const char *ifname, const char *archivo, char *dst, size_t tam)
{
    char ruta[256];
    snprintf(ruta, sizeof ruta, "/sys/class/net/%s/%s", ifname, archivo);
    dst[0] = '\0';
    FILE *f = fopen(ruta, "r");
    if (!f) return;
    if (fgets(dst, (int)tam, f)) dst[strcspn(dst, "\n")] = '\0';
    fclose(f);
}

static void q_red(buf_t *b)
{
    struct ifaddrs *ifs, *i;
    if (getifaddrs(&ifs) != 0) {
        b_printf(b, "(getifaddrs falló: %s)\n", strerror(errno));
        return;
    }
    b_printf(b, "Fuente: getifaddrs() + /sys/class/net\n");
    /* Recorrer interfaces sin repetir nombres */
    char vistos[64][IF_NAMESIZE];
    int nv = 0;
    for (i = ifs; i; i = i->ifa_next) {
        int repetido = 0;
        for (int k = 0; k < nv; k++)
            if (strcmp(vistos[k], i->ifa_name) == 0) repetido = 1;
        if (repetido || nv >= 64) continue;
        snprintf(vistos[nv++], IF_NAMESIZE, "%s", i->ifa_name);

        char mac[32], estado[32], mtu[16];
        leer_sys(i->ifa_name, "address", mac, sizeof mac);
        leer_sys(i->ifa_name, "operstate", estado, sizeof estado);
        leer_sys(i->ifa_name, "mtu", mtu, sizeof mtu);
        b_printf(b, "\n  %s  estado=%s  MAC=%s  MTU=%s\n", i->ifa_name,
                 estado[0] ? estado : "?", mac[0] ? mac : "-", mtu[0] ? mtu : "?");

        for (struct ifaddrs *j = ifs; j; j = j->ifa_next) {
            if (strcmp(j->ifa_name, i->ifa_name) != 0 || !j->ifa_addr) continue;
            int fam = j->ifa_addr->sa_family;
            if (fam != AF_INET && fam != AF_INET6) continue;
            char ip[INET6_ADDRSTRLEN];
            const void *a = fam == AF_INET ? (const void *)&((struct sockaddr_in *)j->ifa_addr)->sin_addr
```

```
                                        : (const void *)&((struct sockaddr_in6 *)j->ifa_addr)->sin6_addr;
            inet_ntop(fam, a, ip, sizeof ip);
            b_printf(b, "      %s  %s/%d\n", fam == AF_INET ? "IPv4" : "IPv6", ip, prefijo_mascara(j->ifa_netmask));
        }
    }
    freeifaddrs(ifs);
}

static void q_procesos(buf_t *b)
{
    int total = 0;
    DIR *d = opendir("/proc");
    if (d) {
        struct dirent *e;
        while ((e = readdir(d)))
            if (isdigit((unsigned char)e->d_name[0])) total++;
        closedir(d);
    }
    b_printf(b, "Procesos en ejecución (directorios numéricos en /proc): %d\n", total);
    b_comando(b, "ps -eo pid,user,%cpu,%mem,stat,comm --sort=-%cpu  (top 20 por CPU)",
              "ps -eo pid,user,%cpu,%mem,stat,comm --sort=-%cpu 2>/dev/null | head -n 21");
}

/* Lista blanca de consultas permitidas */
static const struct {
    const char *clave;
    void (*funcion)(buf_t *);
} CONSULTAS[] = {
    { "cpu", q_cpu },         { "ram", q_ram },         { "disco", q_disco },
    { "fs", q_fs },           { "loadavg", q_loadavg }, { "particiones", q_particiones },
    { "so", q_so },           { "red", q_red },         { "procesos", q_procesos },
};

/* ---------------------------------------------------------------------- */
/* Sockets                                                                */
/* ---------------------------------------------------------------------- */

static int enviar_todo(int fd, const char *buf, size_t len)
{
    while (len > 0) {
        ssize_t n = send(fd, buf, len, MSG_NOSIGNAL);             /* send() */
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        buf += n;
        len -= (size_t)n;
    }
    return 0;
}

static int leer_linea(int fd, lector_t *lr, char *linea, size_t tam)
{
    for (;;) {
        char  *nl = memchr(lr->datos, '\n', lr->len);
        size_t n;
        if (nl)
            n = (size_t)(nl - lr->datos);
        else if (lr->len == sizeof lr->datos)
            n = lr->len;
        else {
            ssize_t r = recv(fd, lr->datos + lr->len,                 /* recv() */
                             sizeof lr->datos - lr->len, 0);
            if (r == 0) return 0;
            if (r < 0) {
                if (errno == EINTR) continue;
                return -1;
            }
            lr->len += (size_t)r;
            continue;
        }
        size_t copiar = n < tam - 1 ? n : tam - 1;
        memcpy(linea, lr->datos, copiar);
        linea[copiar] = '\0';
        size_t consumir = nl ? n + 1 : n;
        memmove(lr->datos, lr->datos + consumir, lr->len - consumir);
        lr->len -= consumir;
        if (copiar > 0 && linea[copiar - 1] == '\r') linea[copiar - 1] = '\0';
        return 1;
    }
}

/* Envía "RESP <n>\n" (o "ERR <n>\n") seguido de los n bytes de la respuesta */
static int enviar_respuesta(int fd, int es_error, const char *texto, size_t n)
{
    char cab[64];
    int l = snprintf(cab, sizeof cab, "%s %zu\n", es_error ? "ERR" : "RESP", n);
    if (enviar_todo(fd, cab, (size_t)l) < 0) return -1;
```

```
    return enviar_todo(fd, texto, n);
}

static int conectar(const char *host, const char *puerto)
{
    struct addrinfo pistas, *res, *p;
    memset(&pistas, 0, sizeof pistas);
    pistas.ai_family   = AF_INET;
    pistas.ai_socktype = SOCK_STREAM;
    int e = getaddrinfo(host, puerto, &pistas, &res);
    if (e != 0) {
        fprintf(stderr, "No se pudo resolver '%s': %s\n", host, gai_strerror(e));
        return -1;
    }
    int fd = -1, err = 0;
    for (p = res; p; p = p->ai_next) {
        fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);   /* (1) socket() */
        if (fd < 0) {
            err = errno;
            continue;
        }
        int flags = fcntl(fd, F_GETFL, 0);
        fcntl(fd, F_SETFL, flags | O_NONBLOCK);
        int r = connect(fd, p->ai_addr, p->ai_addrlen);              /* (2) connect() */
        if (r < 0 && errno == EINPROGRESS) {
            fd_set w;
            FD_ZERO(&w);
            FD_SET(fd, &w);
            struct timeval tv = { .tv_sec = TIMEOUT_CONEXION, .tv_usec = 0 };
            r = select(fd + 1, NULL, &w, NULL, &tv);
            if (r == 0) {
                err = ETIMEDOUT;
                r = -1;
            } else if (r > 0) {
                socklen_t l = sizeof err;
                getsockopt(fd, SOL_SOCKET, SO_ERROR, &err, &l);
                r = err ? -1 : 0;
            } else {
                err = errno;
            }
        } else if (r < 0) {
            err = errno;
        }
        if (r == 0) {
            fcntl(fd, F_SETFL, flags);
            break;
        }
        close(fd);
        fd = -1;
    }
    freeaddrinfo(res);
    if (fd < 0) {
        if (err == ECONNREFUSED)
            fprintf(stderr, "Conexión rechazada: no hay ningún servidor escuchando en %s:%s\n", host, puerto);
        else if (err == ETIMEDOUT)
            fprintf(stderr, "Tiempo de espera agotado: el host %s no responde\n", host);
        else
            fprintf(stderr, "No se pudo conectar: %s\n", strerror(err));
    }
    return fd;
}

static void pedir(const char *mensaje, char *dst, size_t tam, const char *defecto)
{
    printf("%s", mensaje);
    fflush(stdout);
    if (!fgets(dst, (int)tam, stdin)) dst[0] = '\0';
    dst[strcspn(dst, "\r\n")] = '\0';
    if (dst[0] == '\0') snprintf(dst, tam, "%s", defecto);
}

static void log_agente(const char *fmt, ...)
{
    char h[16];
    time_t t = time(NULL);
    struct tm tm;
    localtime_r(&t, &tm);
    strftime(h, sizeof h, "%H:%M:%S", &tm);
    printf("[%s] ", h);
    va_list ap;
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    printf("\n");
    fflush(stdout);
}

int main(int argc, char *argv[])
```

```
{
    char host[256], puerto[16];
    signal(SIGPIPE, SIG_IGN);

    printf("=== Agente de inventario HW/SW - Redes de Datos (UNNE) ===\n");
    if (argc >= 2) snprintf(host, sizeof host, "%s", argv[1]);
    else pedir("IP del servidor de inventario [" HOST_DEFECTO "]: ", host, sizeof host, HOST_DEFECTO);
    if (argc >= 3) snprintf(puerto, sizeof puerto, "%s", argv[2]);
    else pedir("Puerto [" PUERTO_DEFECTO "]: ", puerto, sizeof puerto, PUERTO_DEFECTO);

    char *fin;
    long pn = strtol(puerto, &fin, 10);
    if (*fin != '\0' || pn < 1 || pn > 65535) {
        fprintf(stderr, "Puerto inválido: '%s'\n", puerto);
        return 1;
    }

    printf("Conectando a %s:%s ...\n", host, puerto);
    int fd = conectar(host, puerto);
    if (fd < 0) return 1;

    struct sockaddr_in loc;
    socklen_t ll = sizeof loc;
    getsockname(fd, (struct sockaddr *)&loc, &ll);
    char ip_loc[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &loc.sin_addr, ip_loc, sizeof ip_loc);
    printf("Conexión exitosa: %s:%d (agente) -> %s:%s (servidor)\n", ip_loc, ntohs(loc.sin_port), host, puerto);

    /* Saludo: HOLA <hostname> */
    char nombre[256] = "desconocido", saludo[300];
    gethostname(nombre, sizeof nombre);
    int l = snprintf(saludo, sizeof saludo, "HOLA %s\n", nombre);
    if (enviar_todo(fd, saludo, (size_t)l) < 0) {
        perror("send");
        return 1;
    }

    lector_t lr = { .len = 0 };
    char linea[MAX_LINEA];
    if (leer_linea(fd, &lr, linea, sizeof linea) <= 0 || strncmp(linea, "OK ", 3) != 0) {
        fprintf(stderr, "El servidor rechazó la conexión: %s\n", linea);
        return 1;
    }
    log_agente("Registrado como equipo #%s (%s). Esperando consultas... (Ctrl+C para salir)", linea + 3, nombre);

    int r;
    while ((r = leer_linea(fd, &lr, linea, sizeof linea)) == 1) {
        if (strcmp(linea, "FIN") == 0) {
            log_agente("El servidor finalizó la sesión.");
            close(fd);
            return 0;
        }
        if (strncmp(linea, "CONSULTA ", 9) != 0) {
            const char *msg = "Mensaje no reconocido por el agente\n";
            enviar_respuesta(fd, 1, msg, strlen(msg));
            continue;
        }
        const char *tipo = linea + 9;
        void (*funcion)(buf_t *) = NULL;
        for (size_t i = 0; i < sizeof CONSULTAS / sizeof CONSULTAS[0]; i++)
            if (strcmp(tipo, CONSULTAS[i].clave) == 0) funcion = CONSULTAS[i].funcion;

        buf_t b = { 0 };
        int es_error = 0;
        if (funcion) {
            funcion(&b);
        } else {
            b_printf(&b, "Consulta desconocida: '%s'\n", tipo);
            es_error = 1;
        }
        if (enviar_respuesta(fd, es_error, b.d ? b.d : "", b.len) < 0) {
            free(b.d);
            break;
        }
        log_agente("Consulta '%s' respondida (%zu bytes)", tipo, b.len);
        free(b.d);
    }

    fprintf(stderr, "Se perdió la conexión con el servidor (%s).\n",
            r == 0 ? "el servidor cerró la conexión" : strerror(errno));
    close(fd);
    return 1;
}
```